

고품질 AI 활용을 위한 Agentic Engineering

CONTENTS

목차

CHAPTER 1

Agentic Engineering 소개

CHAPTER 2

Mars Rover 실습 안내

CHAPTER 3

프로젝트 기본 세팅

CHAPTER 4

개발 단계 Phase 수립

CHAPTER 5

개발 시작! (Phase 1)

CHAPTER 6

방향키 개발 (Phase 2)

CHAPTER 7

연료소모량 기능추가 (Phase 3)

CHAPTER 8

미션 (Phase 4)

Chapter 02

Agentic Engineering 소개

SW 품질을 위한 Agentic Engineering

Code Agent를 쓰다 보니까.. 발생하는 문제점들

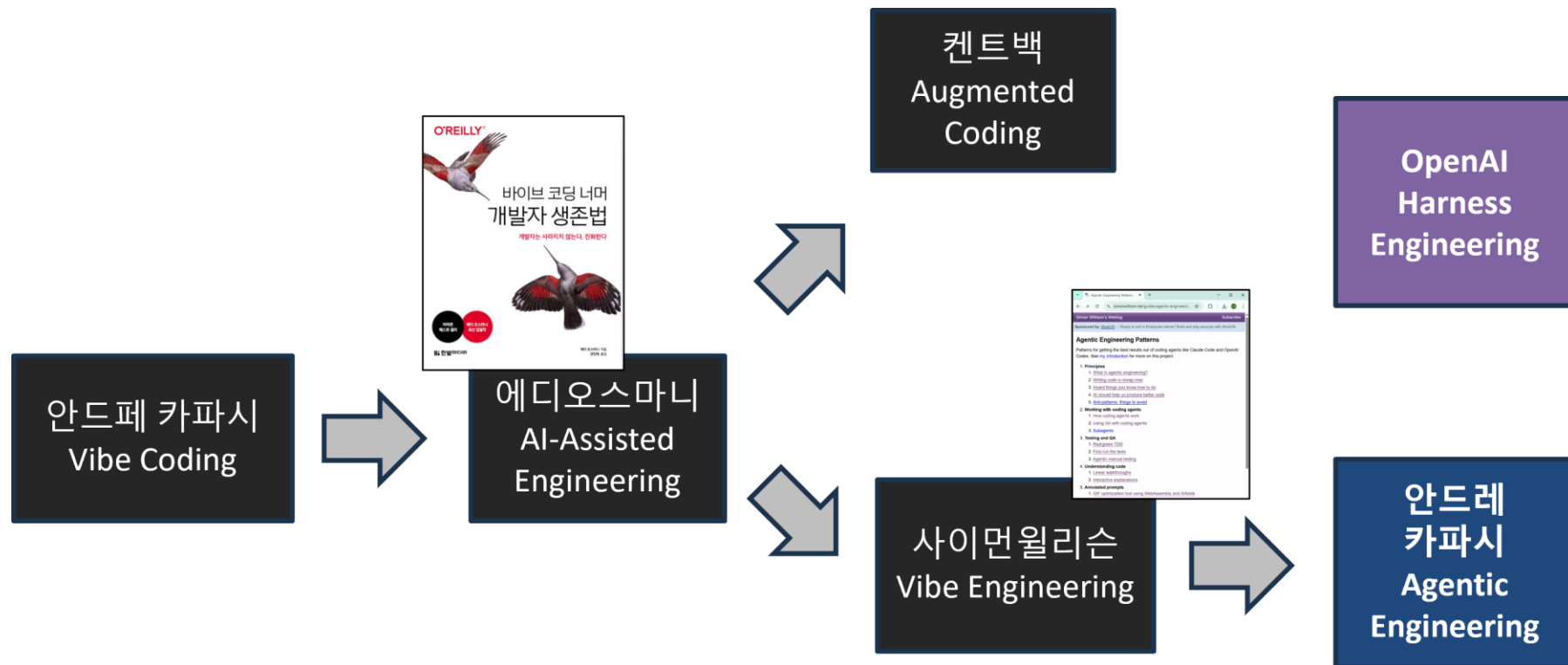
'딸깍'식으로 결과가 빠르게 생성되면서
SW 품질이 쉽게 무너지는 현상이 발생하고 있다.

AI 시대에는 이러한 변화에 맞추어 공학적인 접근이 필요해졌다.

이에 따라 공학적 이론을 기반으로 접근하여,
안전성을 확보하면서도 생산성을 동시에 달성하려는 다양한 방법론들이 등장하기 시작했다.

Agentic Engineering 방법론 등장

에디오스마니, 사이먼 윌리슨, 안드레 카파시 등을 거쳐 방법론이 정리되어 갔다.
Agent를 활용하는 데 공학적 접근 방법(엔지니어링)이 필요하다는 의미에서
Agentic Engineering 으로 이름이 정해졌다. (by 안드레 카파시)



Agentic Engineering 의 시작점

Agentic Engineering은 SW 품질을 높이기 위한 “AI를 활용하는 원칙” 이다.

Agentic Engineering 방법론은 **AI와 사람의 역할을 명확히 구분하는 것으로 시작된다.**

인공지능 (AI)



설계 도움



초안 작성

사람 (Human)



설계 및 기획



최종 승인



리뷰 및 수정

AI를 사용하는 SW 엔지니어들의 미션

‘어떤 코드를 만들 것인가’ 와 ‘어떻게 품질을 검증할 것인가’를 고민해야 한다.

1) Prompt Engineering : LLM에게 어떻게 입력을 잘 줄 것인가에 대한 고민

2) Context Engineering : LLM에게 주는 입력을 어떻게 체계화 할 것인가에 대한 고민

3) Harness Engineering : 안정적으로 SW를 생성해내기 위한 환경을 어떻게 설계할 것인지 고민

- 문제 해결에 필요한 적절한 도구를 선택할 것인지에 대한 고민

4) Agentic Engineering : 최고 수준의 고품질 SW를 만들기 위해 어떻게 협업할지 고민

- 문제를 적절한 수준으로 구체화하고, 세분화한다. (문제 분해 능력)
- 생성된 결과를 반드시 사람이 검증한다. (코드 리뷰 능력)

“바이브코딩” vs Agentic Engineering

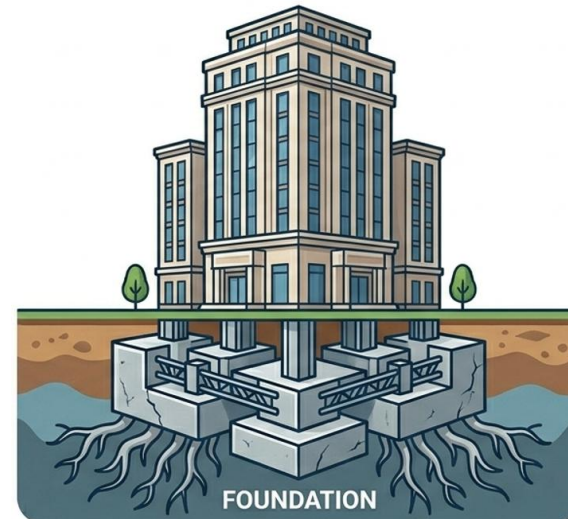
바이브코딩은 단순히 LLM으로 코드를 생성하는 모든 행위를 뜻하지 않는다.

‘코드 존재를 잊어버린다’ 라는 것에 집중한 용어이다.

Agentic Engineering에서는 이해가 가지 않는 코드를 사용하면 안 된다.

바이브 코딩과 다르게, 어떠한 코드가 만들어졌는지 AI와 리뷰를 해야 한다.

바이브 코딩
으로 만든 코드



Agentic Engineering
으로 만든 코드

Model이 발전하면 구식이 되는 이론?

Harness Engineering은 Model이 발전할수록 덜어내야 하는 경향이 있지만,
Agentic Engineering은 시간이 지나도 구식이 되는 이론이 아니다.

모델의 성능이 아무리 발전하더라도,
사람이 직접 품질과 방향을 통제하며,
최종 책임까지 지는 방향성은 모델이 아무리 발전해도 변하지 않는다.

지금부터 Agentic Engineering 교육은 역할 분리에 집중한다.

사람이 할 일과 AI가 할 일을 명확히 구분하고, 사람은 자신의 역할에 집중하여 진행한다.

고품질의 소프트웨어를 만들기 위해,

어떤 일을 사람이 해야 하는지,
어떤 일을 AI가 담당해야 하는지를 구분하며 실습에 참여하자.

Chapter 01

Mars Rover 실습 안내

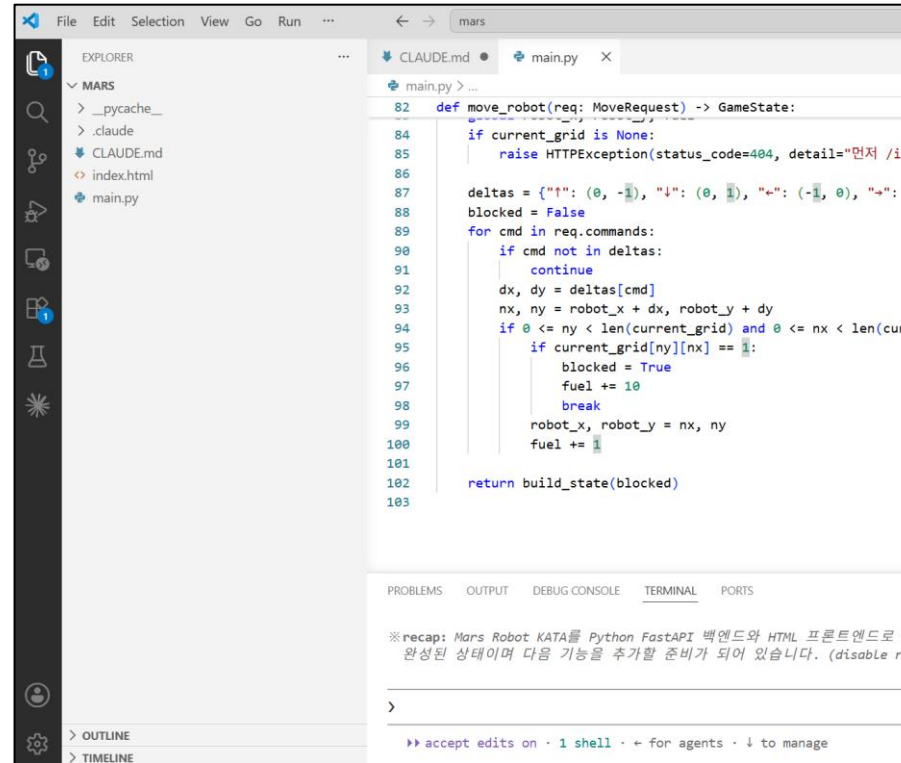
Agentic Engineering 실습

Mars Rover라는 훈련자료를 통해, Agentic Engineering을 경험한다.

높은 품질의 SW를 개발하기 위한 방법론인 Agentic Engineering 경험
AI의 역할은 초안 개발 / 인간은 설계 및 리뷰 진행

커리큘럼

1. 사전 세팅
2. 프레임워크 세팅
3. Agentic Engineering을 적용한 기능 개발



```
82 def move_robot(req: MoveRequest) -> GameState:
83     if current_grid is None:
84         raise HTTPException(status_code=404, detail="먼저 /init
85     deltas = {"↑": (0, -1), "↓": (0, 1), "←": (-1, 0), "→": (1, 0)}
86     blocked = False
87     for cmd in req.commands:
88         if cmd not in deltas:
89             continue
90         dx, dy = deltas[cmd]
91         nx, ny = robot_x + dx, robot_y + dy
92         if 0 <= ny < len(current_grid) and 0 <= nx < len(current_grid[ny]):
93             if current_grid[ny][nx] == 1:
94                 blocked = True
95                 fuel += 10
96                 break
97             robot_x, robot_y = nx, ny
98             fuel += 1
99     return build_state(blocked)
```

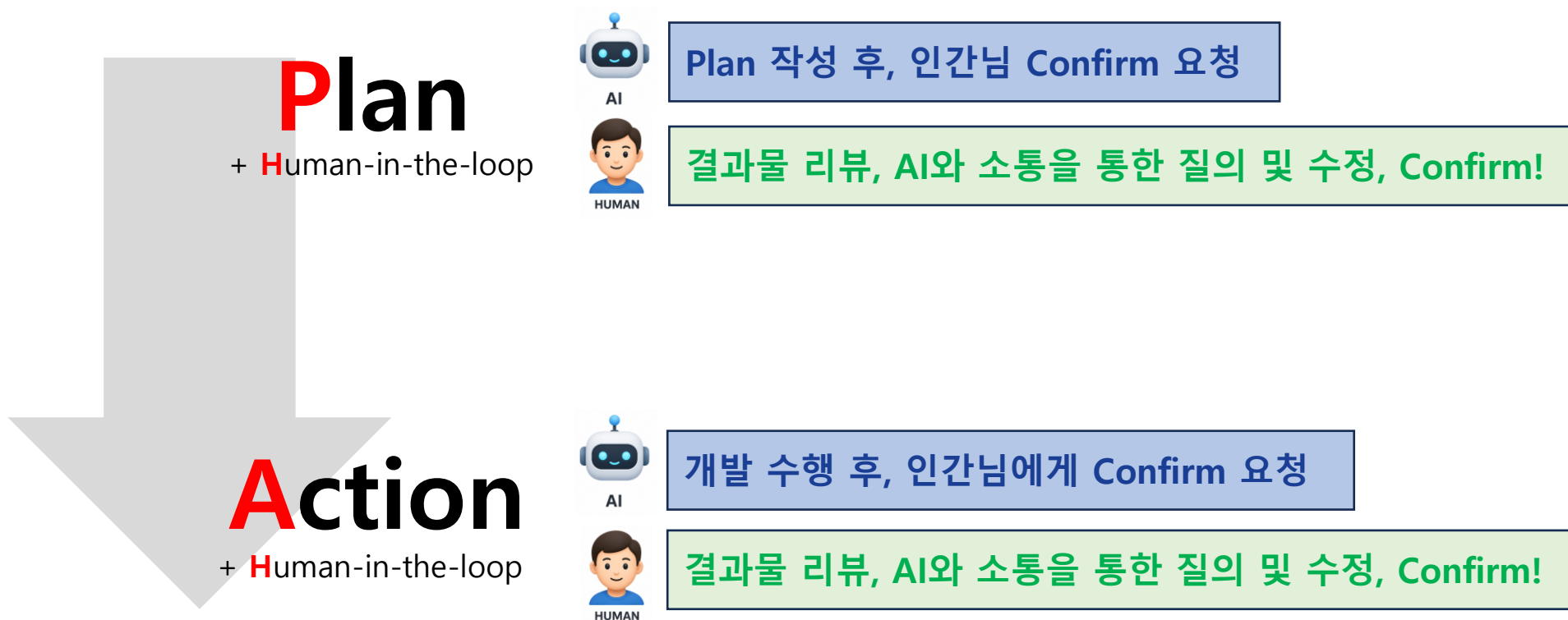
※ recap: Mars Robot KATA를 Python FastAPI 백엔드와 HTML 프론트엔드로 구현된 상태이며 다음 기능을 추가할 준비가 되어 있습니다. (disable recap)



Mars Rover KATA를 통한
Agentic Engineering 실습

목적

이번 실습은 Agentic Engineering은 PH - AH Loop를 반복하는 것이 목적이다.



Mars Rover란?

Mars Rover(화성탐사) KATA(훈련자료) 소개

TDD 진영에서 유명한 훈련 자료를 각색함.

2 x 2 Grid 맵에서, 우주선 한 대가 존재한다.
벽은 1로 되어있고, 이동 가능한 곳은 0 이다.

우측 예시에서

→ ↑ 방향키 명령어를 전송하면,
우주선이 (2,1) 에서 (1,2)로 이동한다.

만약 →→ 명령을 전송하면,
벽에 부딪혀 WARNING 이 뜨며,
(2,1) 에서 (2,2)에서 멈춘다.



Mars Rover란?

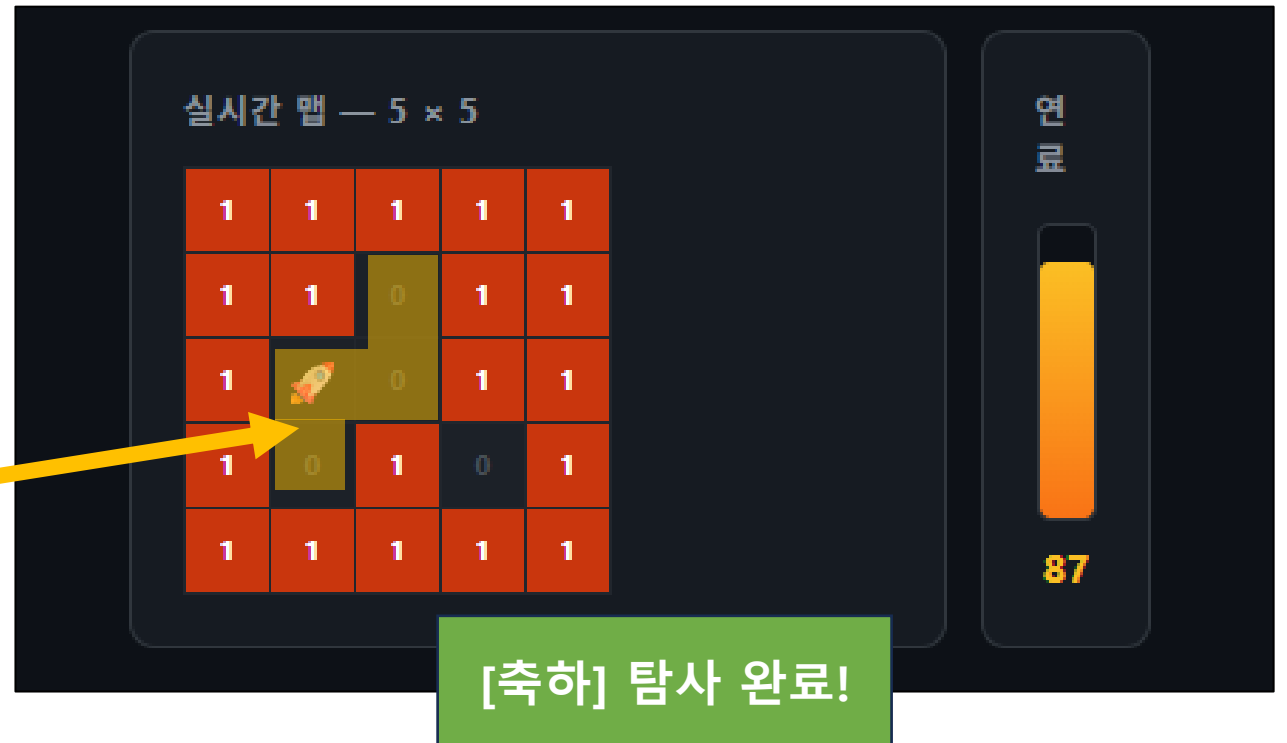
Mars Rover(화성탐사) KATA(훈련자료) 소개

한 번 움직일 때 마다 연료소모량이 1 증가하고,
부딪히면 10 이 올라간다.

연료소모량 이내, 움직일 수 있는 모든 곳을 방문했다면,
탐사 성공이다.

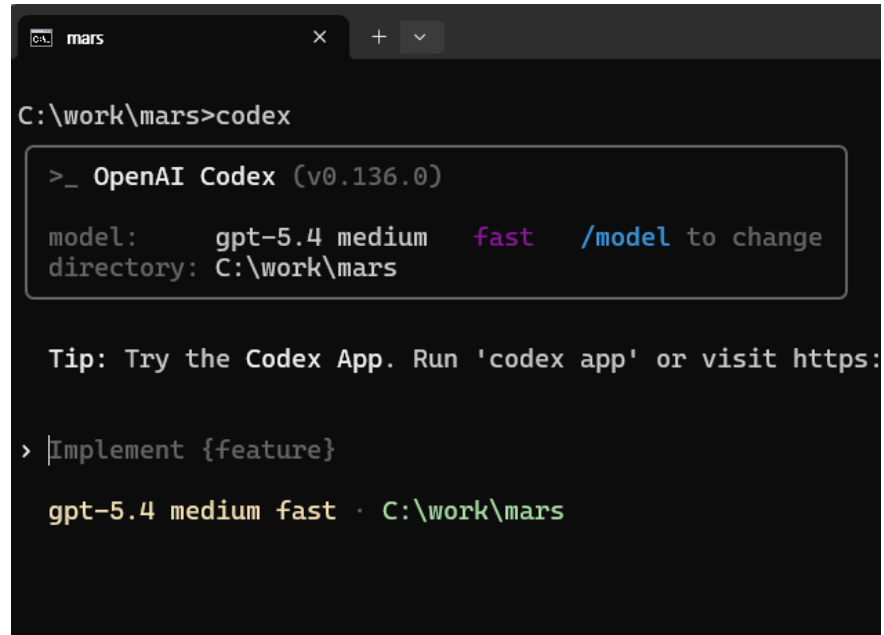
연료소모량이 100이 될 때까지
모든 곳을 탐사하지 못했다면,
탐사 실패이다.

탐사 가능한 위치



실습 환경

- 1) Codex, Claude Code를 CLI 기반으로 사용
- 2) vscode (Visual Studio Code)를 개발 용도로 사용
프로그래밍 용도로 사용 예정



```
C:\work\mars>codex

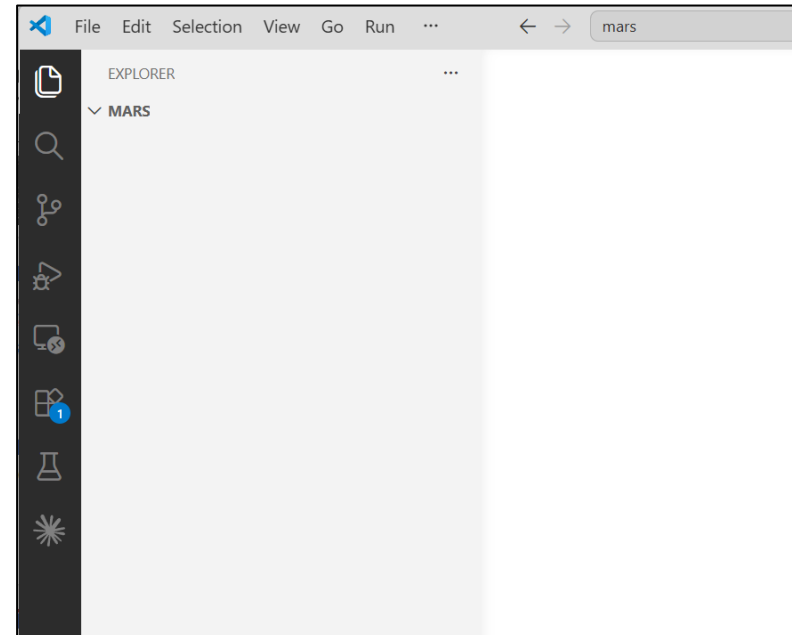
>_ OpenAI Codex (v0.136.0)

model:      gpt-5.4 medium  fast  /model to change
directory:  C:\work\mars

Tip: Try the Codex App. Run 'codex app' or visit https:

> [Implement {feature}]

gpt-5.4 medium fast · C:\work\mars
```



Python 으로 Frontend / Backend 개발을 진행

ABAP / Java 등 타 언어 SW 엔지니어들도 쉽게 이해할 수 있는 언어를 선택

Frontend

- HTML / CSS / JavaScript
- **Frontend는 관심사가 아님 --> 코드리뷰를 하지 않는다!**

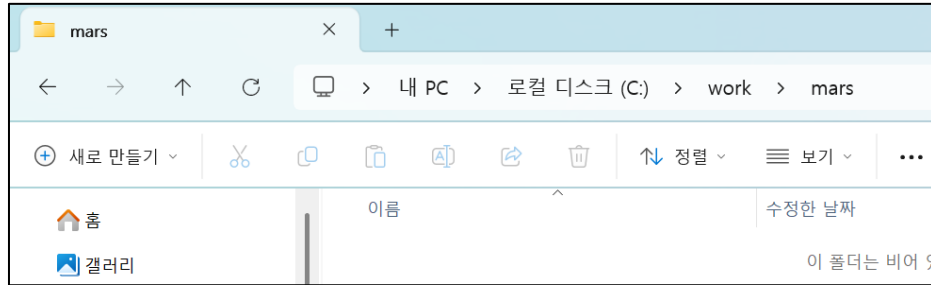
Backend

- Python (FastAPI Framework)
- **코드리뷰를 진행함 (Agentic Engineering 진행)**

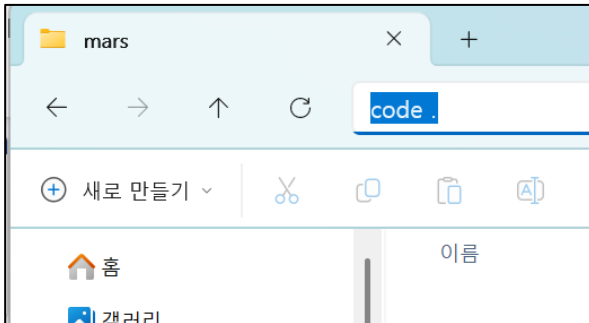
vscode 세팅

폴더 생성 후, **code .** 입력하여 vscode 실행
c:\work\mars 폴더 생성 후, vscode 열기

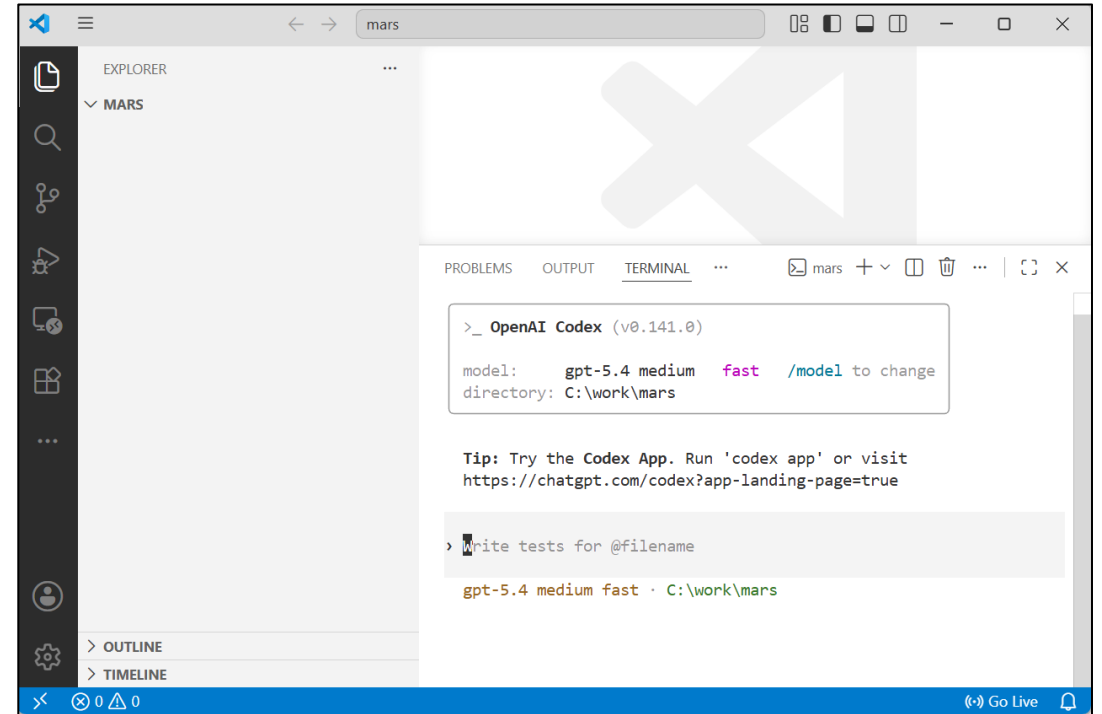
폴더 생성 후 진입



code . 입력



터미널을 열어 codex 실행



동작모드 변경 - 기본 모드 설정

Codex 사용 시 권한을 자주 물어보지 않도록 세팅

```
> /per  
  
/permissions choose what Codex is allowed to do  
/personality choose a communication style for Codex
```

/permissions 입력

```
Update Model Permissions  
  
1. Read Only  
  
2. Ask for approval (current)  
  
3. Approve for me  
> 4. Full Access
```

Full Access 선택 후 엔터

```
Enable full access?  
When Codex runs with full access, it will ask for approval.  
Exercise caution with full access to avoid unexpected behavior.  
  
1. Yes, continue anyway  
> 2. Yes, and don't ask again  
3. Cancel  
  
Press enter to confirm or escape to cancel
```

더 이상 묻지 않기

더블체크

GPT-5.4, medium, fast 세팅이 잘 되어있는지 확인하기

빠른 실습을 위해 GPT-5.5가 아닌 5.4 + medium + /fast가 맞는지
더블 체크해주세요.

```
PS C:\work\mars> codex

>_ OpenAI Codex (v0.141.0)

model: gpt-5.4 medium fast /model to change
directory: C:\work\mars

Tip: Try the Codex App. Run 'codex app' or visit https://

> Write tests for @filename

gpt-5.4 medium fast · C:\work\mars
```

Chapter 02

프로젝트 기본 세팅

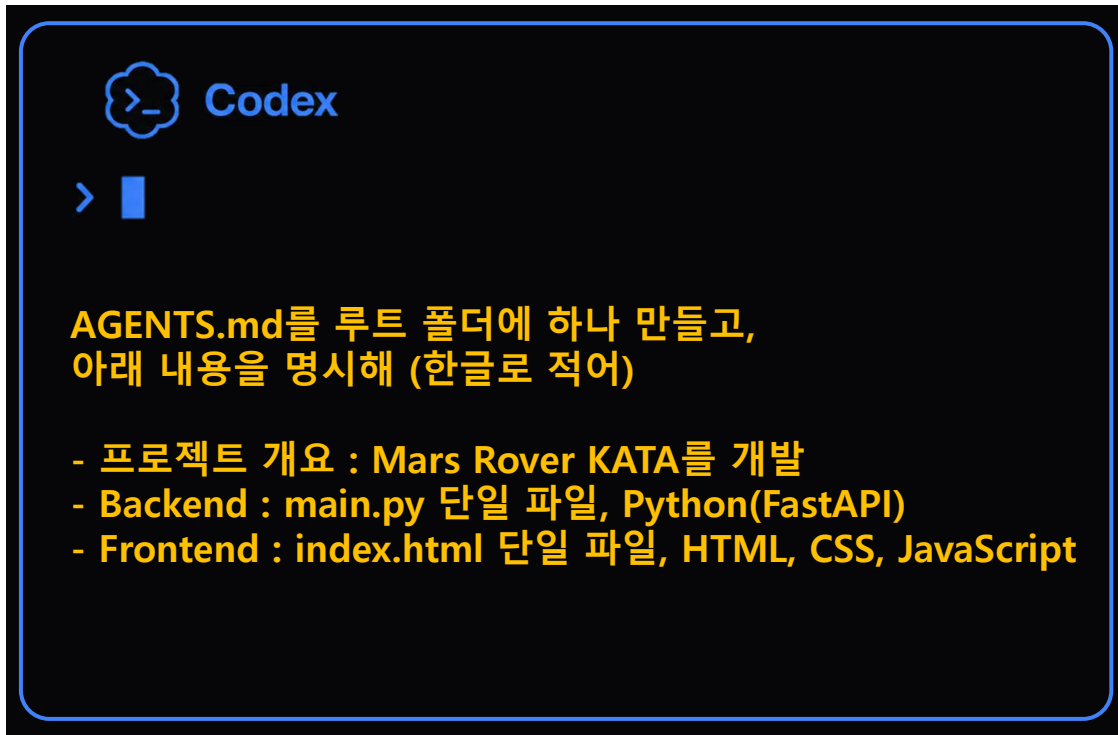
Agentic Engineering 실습

AGENTS.md 파일 생성

1. 빈 AGENTS.md 파일 생성

Codex에서 사용되는 규칙 / 지침 파일

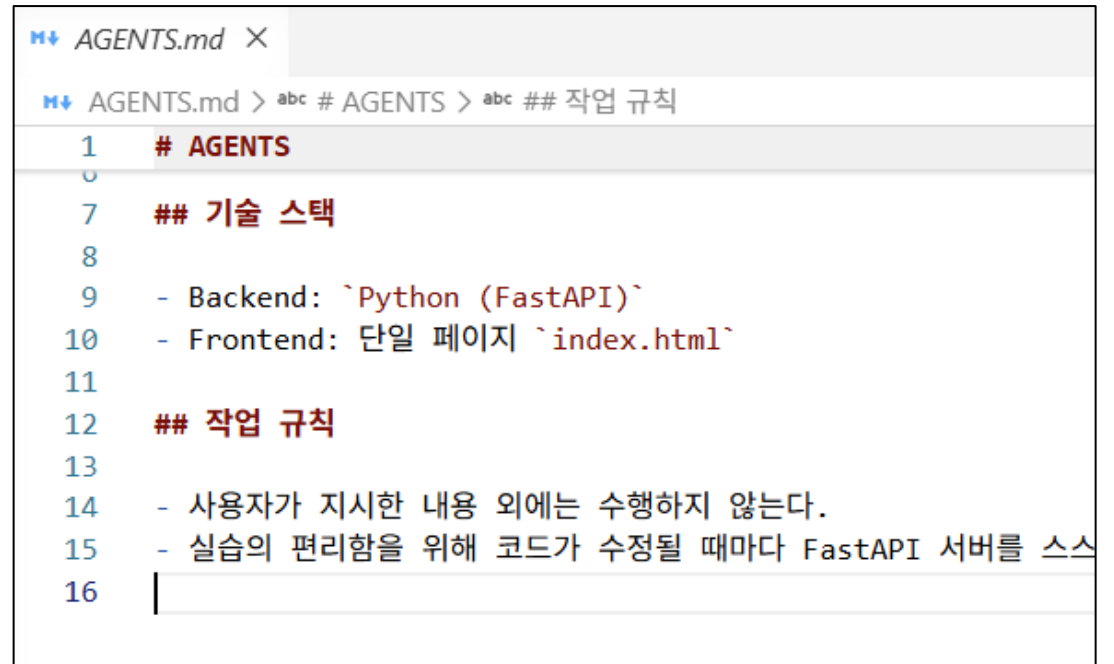
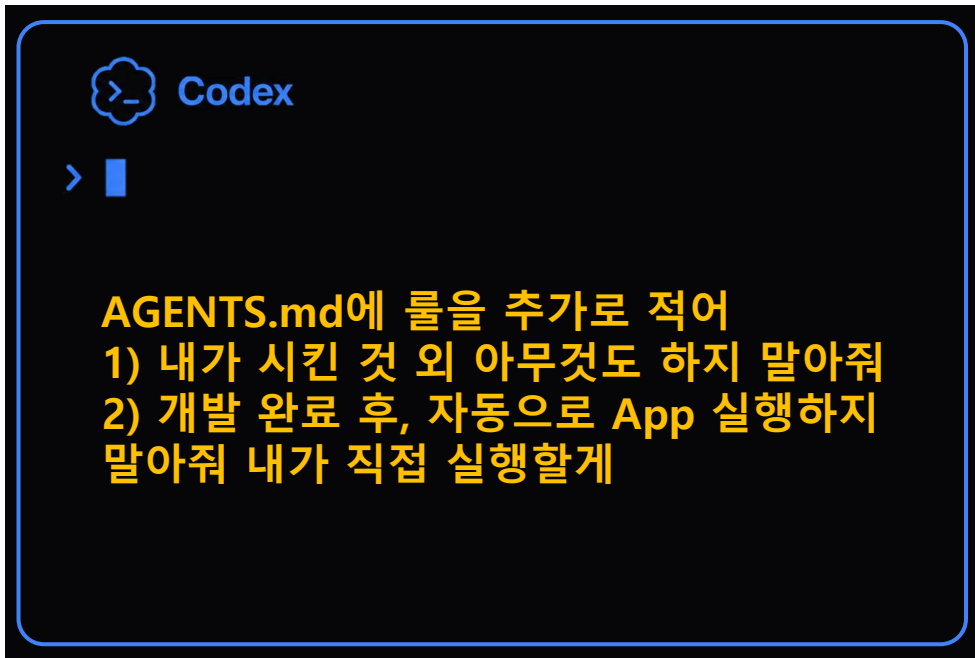
이곳에 내용을 기입하면, Codex는 사용자의 규칙(지침)으로 간주하고 행동한다.



AGENTS.md 에 Rule 추가하기

2. Codex에게 유의사항 추가

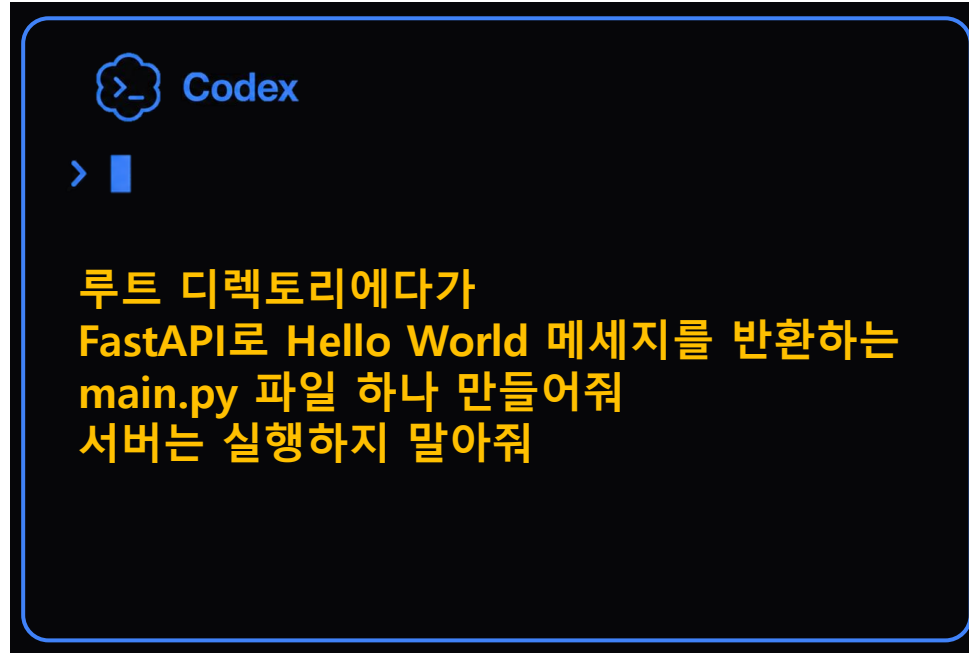
- AGENTS.md에게 Rule을 알려준다.



최소한의 프로젝트 생성하기

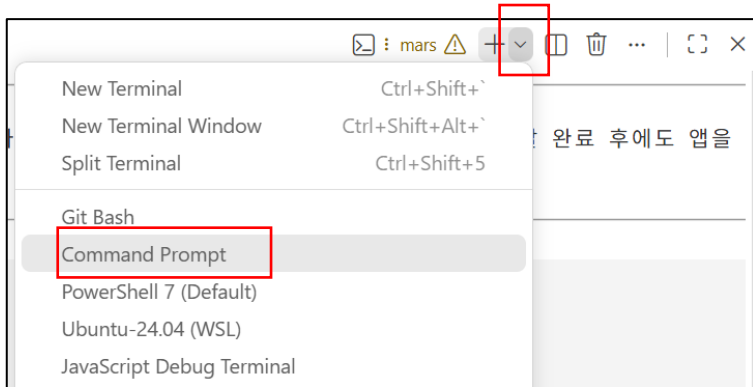
3-1. 최소한의 프레임워크 준비하기 (Backend - FastAPI)

- 최소한의 Backend 서버 준비 및 실행



프로젝트 준비

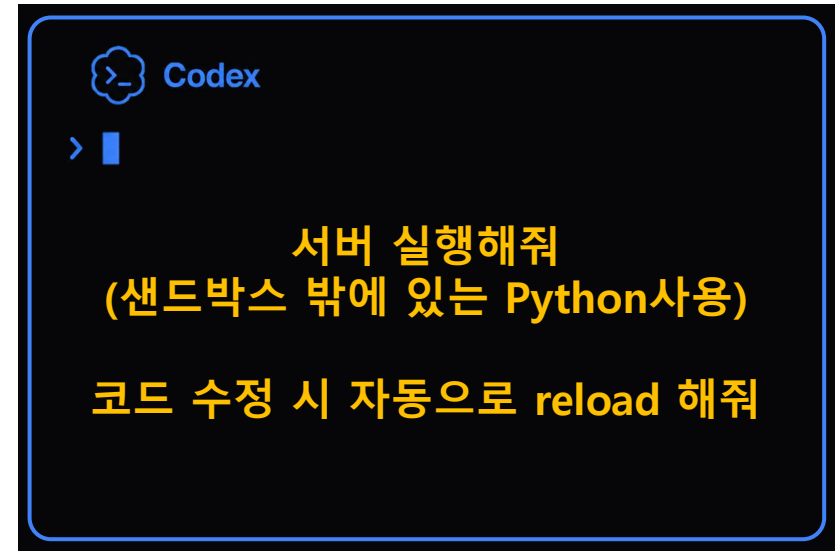
3-2. 서버 실행용 (App 실행용) Codex 를 켜다.



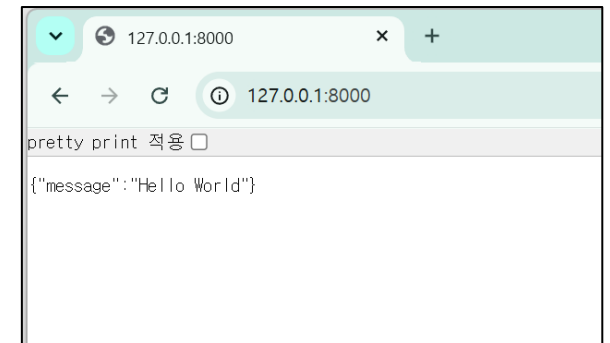
명령프롬프트 하나 더 띄우기



Codex 실행



서버 실행 후
크롬에서 결과 확인
(이 크롬 창은 닫지 않는다.)



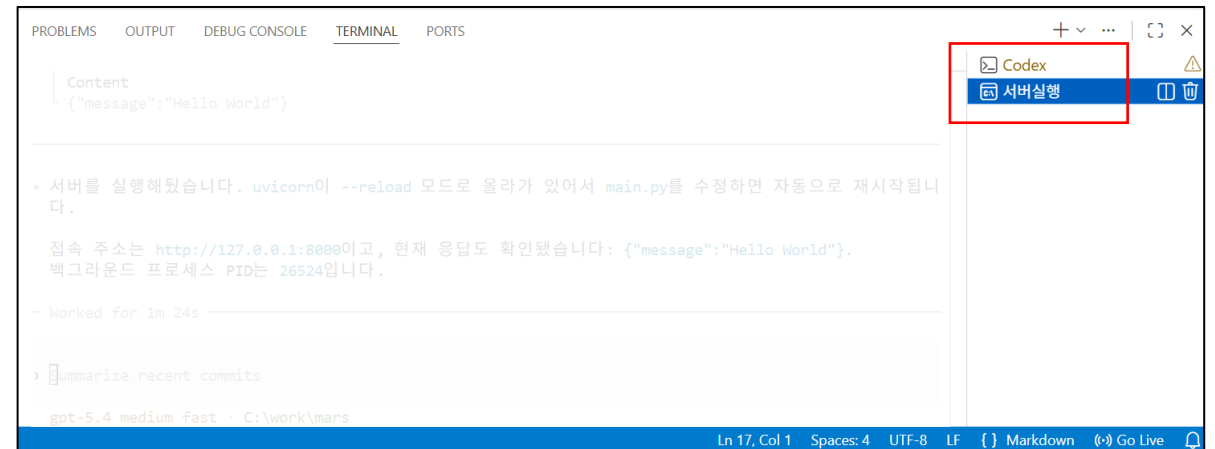
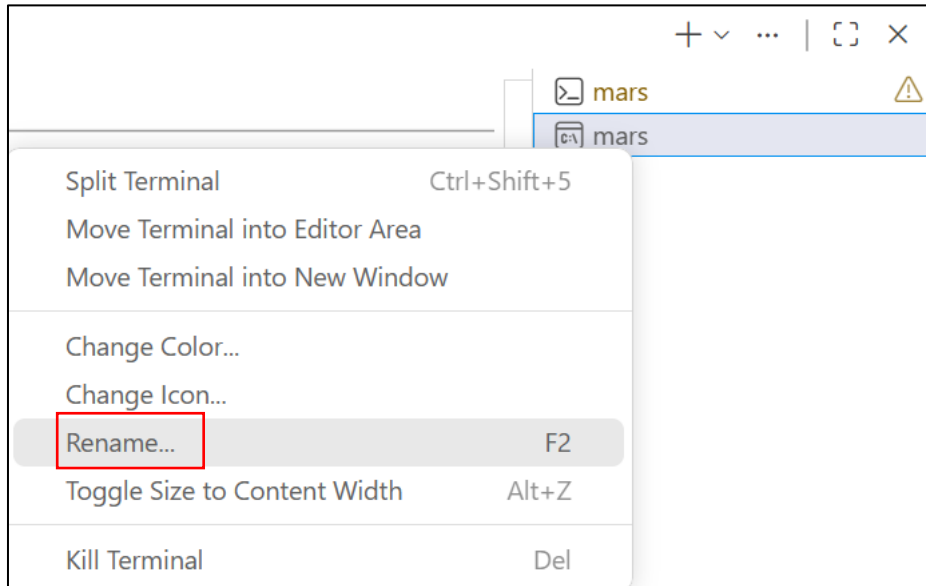
Agent의 랜덤성으로 인해, 실제 실행 결과는 이미지와 다를 수 있습니다.
위 실행 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

프로젝트 준비

3-3. 터미널 이름을 변경해두자.

마우스 우측 버튼 > Rename

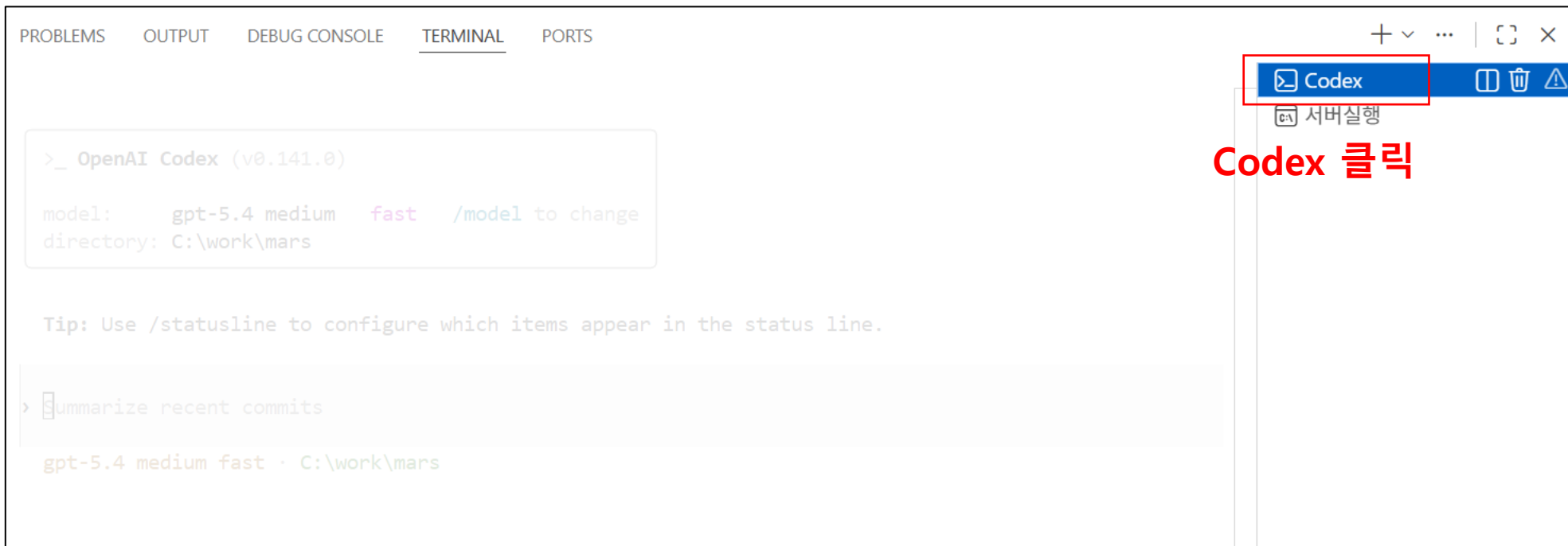
- Codex
- 서버실행



Agent의 랜덤성으로 인해, 실제 실행 결과는 이미지와 다를 수 있습니다.
위 실행 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

3-4. Codex 개발 이어서 하기

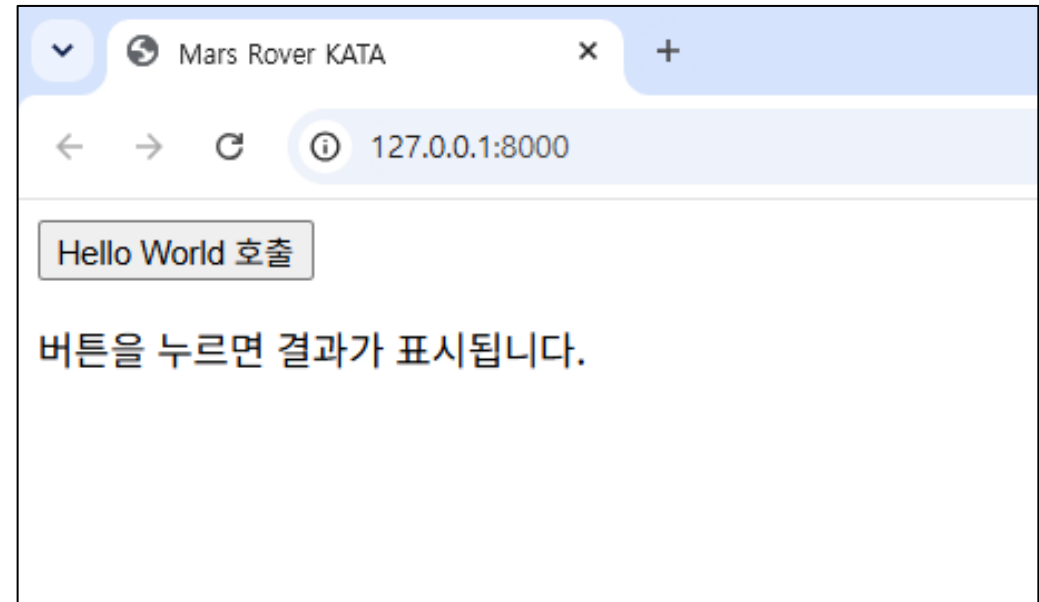
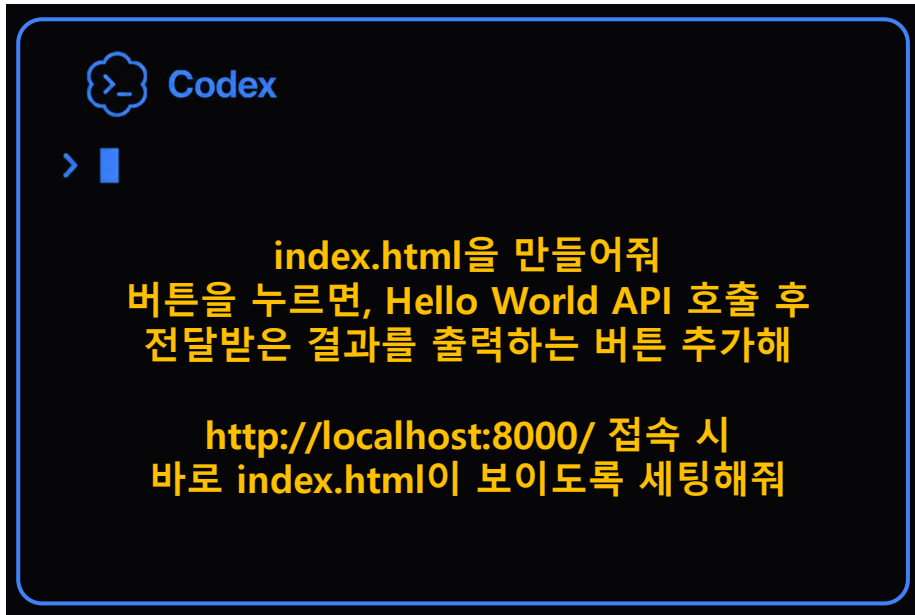
Codex 터미널을 선택하여, 개발을 이어서 하자.



최소한의 프로젝트 생성하기

4. 최소한의 프레임워크 준비하기 (Frontend - HTML/CSS/JavaScript)

- 최소한의 Frontend 준비 및 실행



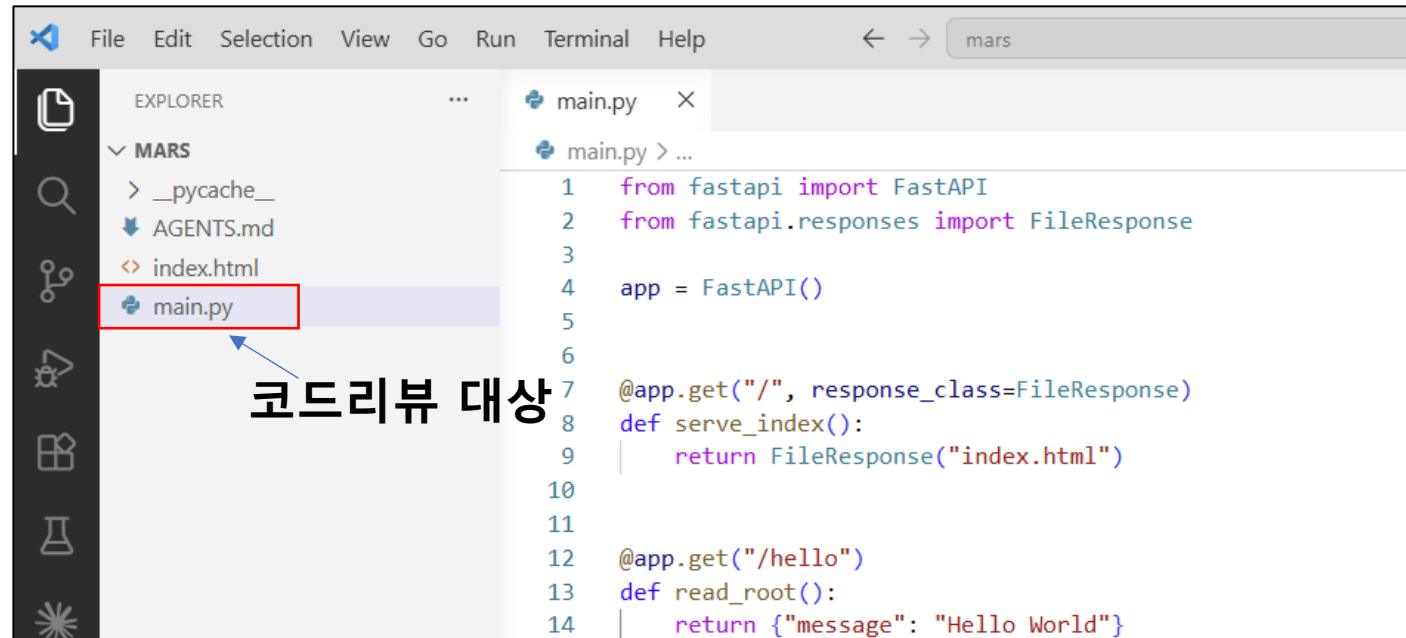
main.py는 앞으로 코드리뷰 대상이다.

Frontend는 코드리뷰 하지 않는다.

- index.html 은 코드리뷰 하지 않는다. 관심사가 아니다.

Backend는 코드 리뷰를 한다.

- main.py 단일 파일 사용 예정
- Agentic Engineering 적용 대상
- 이해하지 못한 코드는 사용하지 않는 것이 원칙



[복습] Agentic Engineering 의 Cycle

Agentic Engineering 은 PH - AH Loop를 반복한다.

Harness와 Context Engineering 내용은 생략하여 실습이 진행된다.



[참고] Agentic Engineering 의 Cycle의 확장 (AI + TDD)

Test 코드까지 추가된 실습은 하지 않는다.



Chapter 03

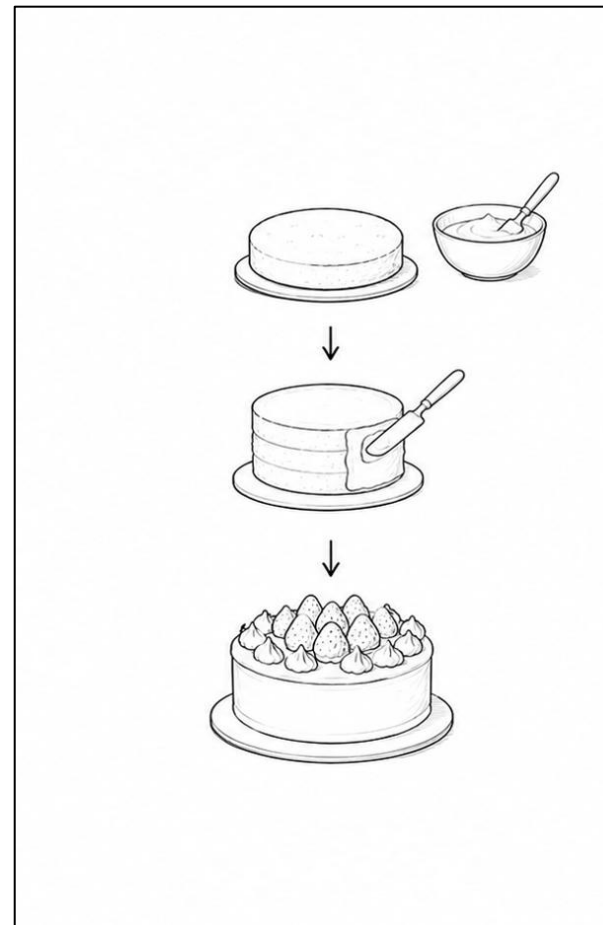
개발 단계 Phase 수립

Agentic Engineering 실습

전체 개발 단계 전략을 세운다.

예시) 빵의 개발 단계

- Phase 1 : 빵 모듈, 크림 준비
- Phase 2 : 빵 쌓고 크림 바르기
- Phase 3 : 데코레이션하기



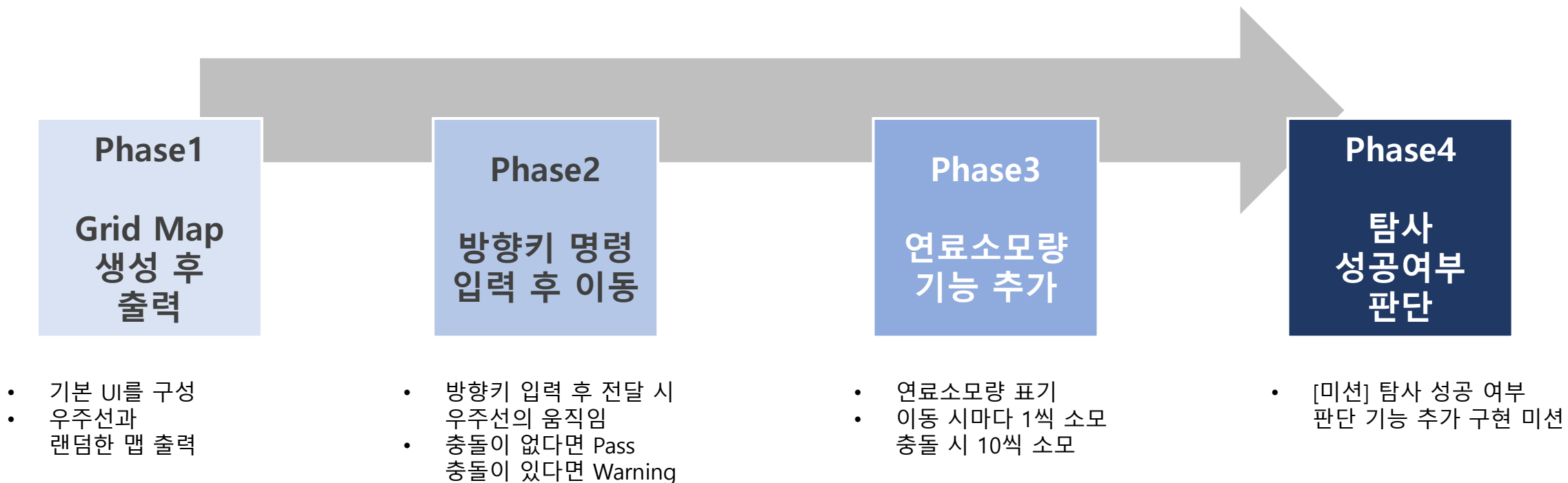
케이크를 만들기 위한
3 단계 Phase 예시

Phase 작성하기

개발 단계를 작성한다.

단계별 어떤 내용을 개발할지 계획을 잡는다.

각 Phase별로, 동작되어야 하며, 사람이 직접 테스트가 가능해야 한다.



5. Phase 전체 개발 단계 전략 수립

전체 개발 단계를 작성한다.



- > ■ docs/PHASES/PHASE1 ~ 4.md 파일에
Phase1 ~ 4까지 아래 내용 각각 써줘
그리고 AGENTS.md에 PHASES 폴더를 명시해줘

=====

Phase1 : Grid Map 생성 후 출력
- 기본 UI 구성, 우주선과 랜덤한 맵 출력

Phase2 : 방향키 명령 입력 후 이동
- 방향키 입력 후 전달 시
- 우주선의 움직임
- 충돌이 없다면 Pass
- 충돌이 있다면 Warning

Phase3 : 연료소모량 기능 추가
- 연료소모량 표기
- 이동 시마다 1씩 소모
- 충돌 시 10씩 소모

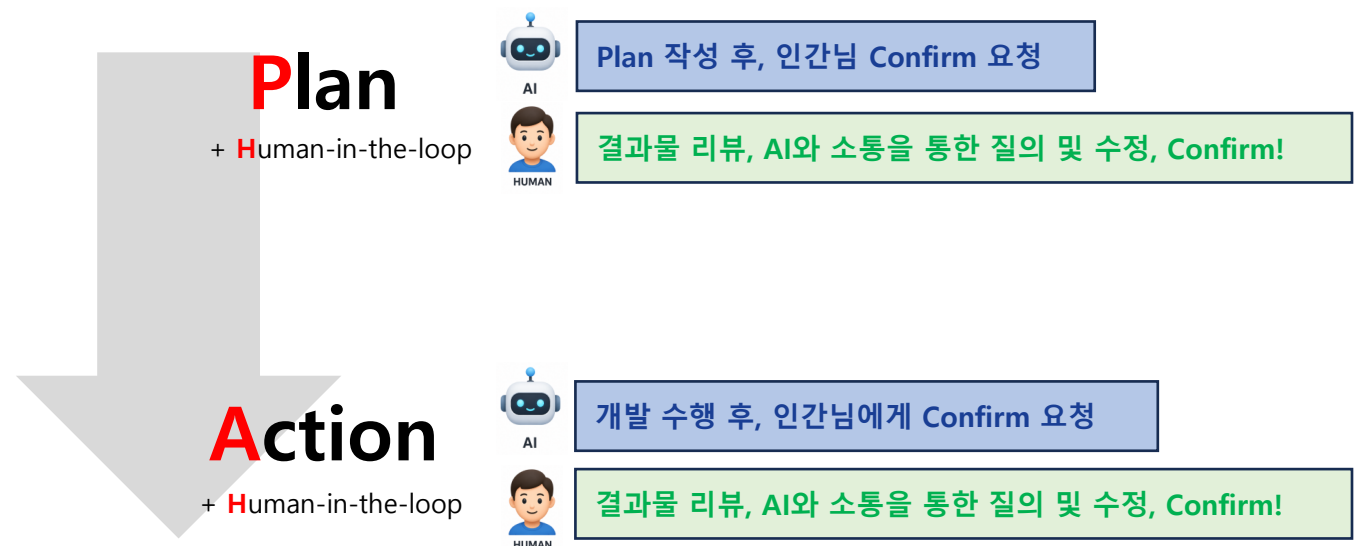
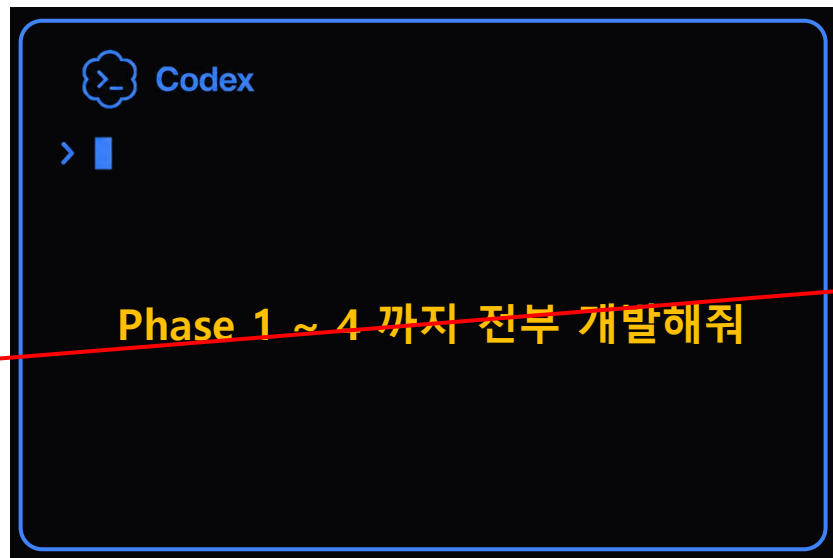
Phase4 : 탐사 성공 여부 판단
- 움직일 수 있는 곳을 모두 탐사하면, 미션 성공 메시지

유의사항. 하면 안 되는 것!

AI에게 모든 것을 맡기면 안 된다.

지금 작성한 것은 전체 개발 과정의 Blueprint 이다.
앞으로 단기 플랜인 PLAN.md를 작성하고 한 단계씩 구현할 예정이다.

인간이 이해한 소스코드를 사용하는 것이 고품질 SW를 만드는 첫 단계이다.
꼼꼼한 코드 리뷰와 함께하는 AI 구현을 실습한다.



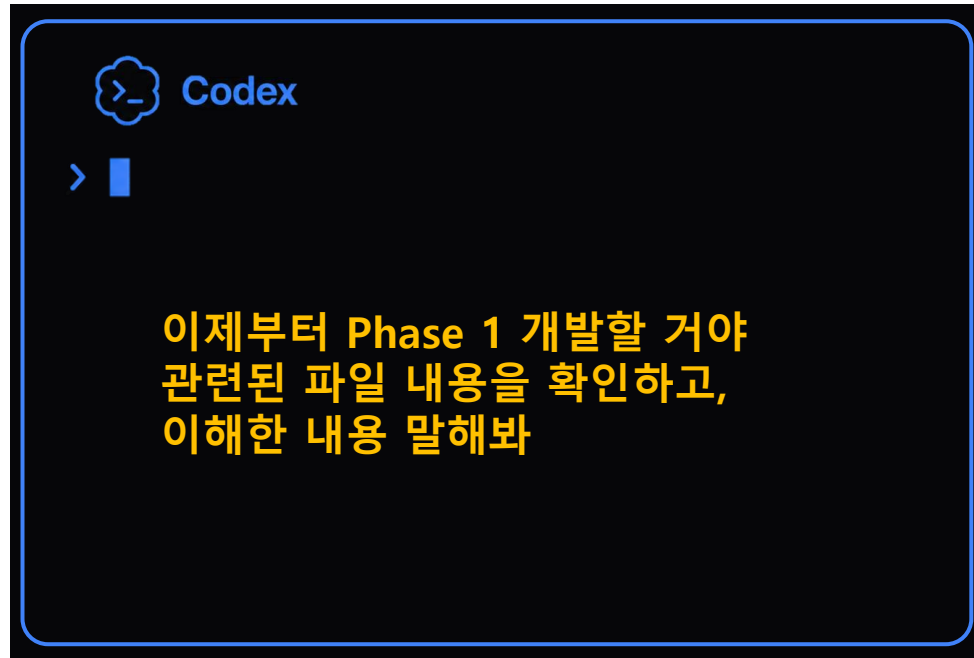
Chapter 04

개발 시작! (Phase 1)

Agentic Engineering 실습

Phase 1 이해시키기

6. Phase 1을 시작한다고 알리기
- Phase 1에 대한 내용을 물어본다.



7. Init 에 대한 동작 Plan 작성을 지시한다.



프로젝트 루트 폴더에 PLAN.md 파일을 하나 만들고
에다가 다음 기능 구현에 대한 플랜을 작성해 보

1) 프론트에서 height, width 입력받고 init 버튼 누르면
백엔드에서 grid 배열에 세팅되는 거야.

2) 그리고 0과 1로 된 랜덤한 배열이 채워지고,
배열이 반환되면 프론트에서 맵이 출력되는거야

```
PLAN.md x
PLAN.md > abc # Phase 1 구현 플랜 > abc ## 구현 흐름 > abc ### 3. Grid 생성 로직 구현

1  # Phase 1 구현 플랜
2
3  ## 목표
4
5  - 프론트에서 `height`, `width` 값을 입력받는다.
6  - `init` 버튼 클릭 시 백엔드로 맵 생성 요청을 보낸다.
7  - 백엔드는 입력값 기준으로 `grid` 배열을 생성한다.
8  - `0`과 `1`로 이루어진 랜덤한 2차원 배열을 채운 뒤 반환한다.
9  - 프론트는 응답받은 배열을 화면에 맵 형태로 출력한다.
10
11 ## 구현 흐름
12
13 ### 1. 프론트 입력 UI 구성
14
15 - `index.html`에 `height`, `width` 입력 필드를 추가한다.
16 - `init` 버튼을 추가한다.
17 - 버튼 클릭 시 입력값을 읽어서 API 요청을 보내는 스크립트를 작성한다.
18
19 ### 2. 백엔드 API 설계
20
21 - FastAPI에 맵 초기화용 엔드포인트를 추가한다.
22 - 요청값으로 `height`, `width`를 받는다.
23 - 입력값이 1 이상의 정수인지 검증한다.
24
```

8. Plan 내용을 완벽하게 이해한다.

이해가 가지 않는 내용에 대해 AI와 토론을 한다.

AI에게 수정을 지시한다.



버튼의 이름을
"탐사준비" 로 Plan 수정해줘

```
PLAN.md x
PLAN.md > abc # Phase 1 구현 플랜 > abc ## 구현 흐름 > abc ### 3. Grid 생성 로직 구현

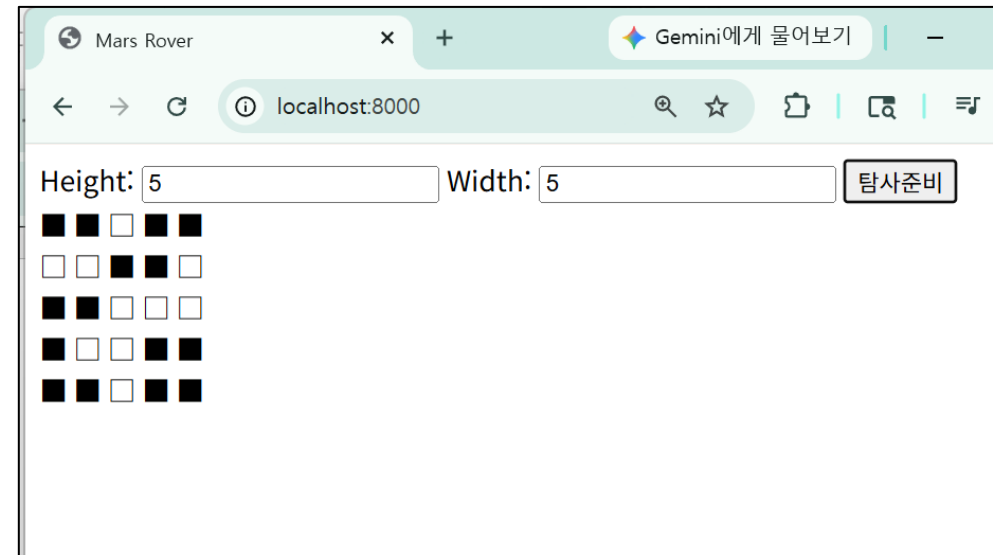
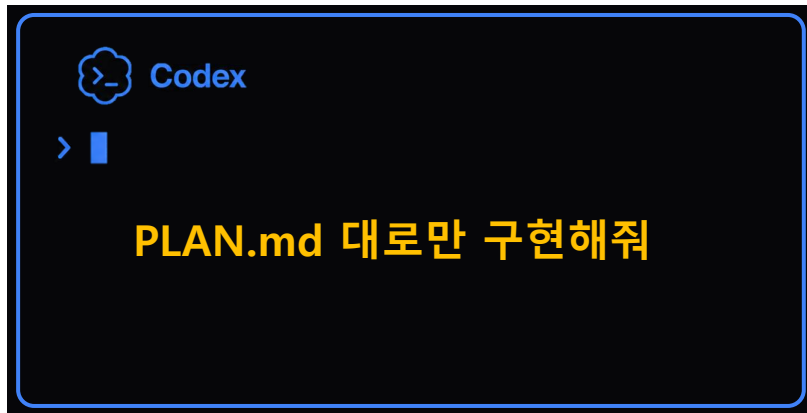
1  # Phase 1 구현 플랜
2
3  ## 목표
4
5  - 프론트에서 `height`, `width` 값을 입력받는다.
6  - `init` 버튼 클릭 시 백엔드로 맵 생성 요청을 보낸다.
7  - 백엔드는 입력값 기준으로 `grid` 배열을 생성한다.
8  - `0`과 `1`로 이루어진 랜덤한 2차원 배열을 채운 뒤 반환한다.
9  - 프론트는 응답받은 배열을 화면에 맵 형태로 출력한다.
10
11 ## 구현 흐름
12
13 ### 1. 프론트 입력 UI 구성
14
15 - `index.html`에 `height`, `width` 입력 필드를 추가한다.
16 - `init` 버튼을 추가한다.
17 - 버튼 클릭 시 입력값을 읽어서 API 요청을 보내는 스크립트를 작성한다.
18
19 ### 2. 백엔드 API 설계
20
21 - FastAPI에 맵 초기화용 엔드포인트를 추가한다.
22 - 요청값으로 `height`, `width`를 받는다.
23 - 입력값이 1 이상의 정수인지 검증한다.
24
```


지금까지 동작

지금까지 Plan 에 대한 검토를 하였다.
이제 Action을 지시하면 된다.



9. Plan 대로 구현을 지시한다.



10. 코드 리뷰

- 이해하지 못하는 코드가 적용되어서는 안 된다.
- 코드를 꼼꼼히 리뷰하고, 수정이 필요한 부분은 수정을 요청한다.



init 코드가 이해가 안 가는데
쉽게 설명하고, 주석도 달아줘

```
main.py ×
main.py > ...

1  import random
2  from fastapi import FastAPI
3  from fastapi.responses import FileResponse
4  from pydantic import BaseModel
5
6  app = FastAPI()
7
8
9  class InitRequest(BaseModel):
10     height: int
11     width: int
12
13
14  @app.get("/", response_class=FileResponse)
15  def serve_index():
16     return FileResponse("index.html")
17
18
19  @app.post("/init")
20  def init_grid(req: InitRequest):
21     grid = [[random.randint(0, 1) for _ in range(req.width)] for _ in range(req.height)]
22     return {"grid": grid}
23
```

지금까지 동작

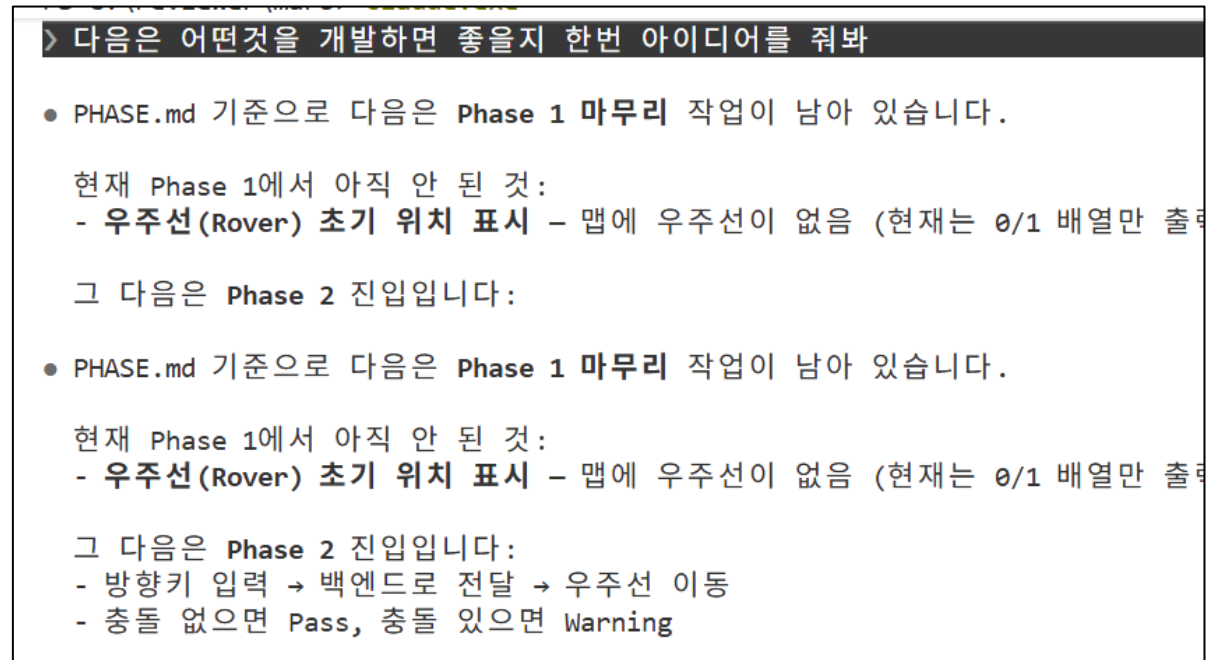
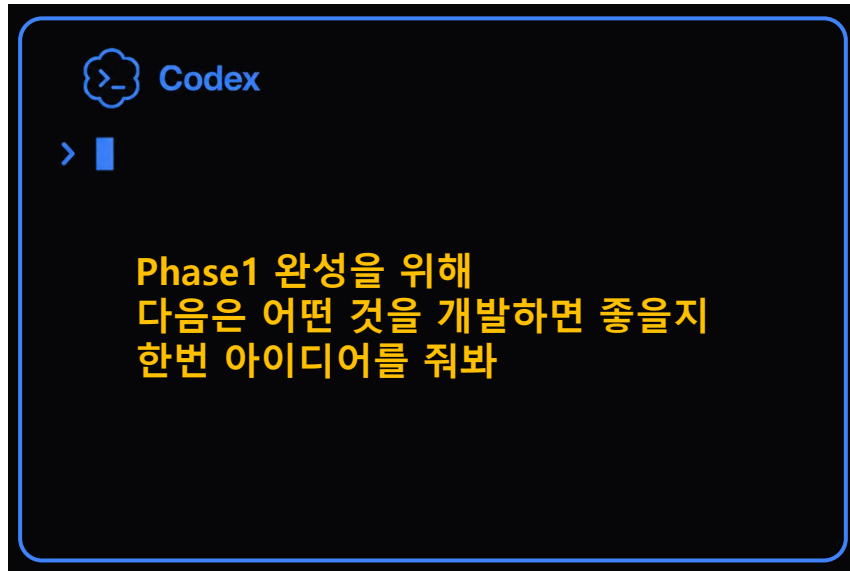
하나의 Cycle이 완료되었다.

이제 다음 어떤 동작을 개발할지 고민해보자.



Agent의 랜덤성으로 인해, 실제 실습 결과는 이미지와 다를 수 있습니다.
위 실습 예시와 동일하게 동작하지 않을 수 있음을 참고하시기 바랍니다.

11. AI와 대화를 통해, 다음 개발에 대한 방향성에 대한 고민을 해본다.



12. Plan 작성 후, Plan 검토를 한다.

반드시 PLAN 내용을 모두 이해한다.

내가 생각하고 있는 PLAN 방향성이 맞는지 한번 더 고민해본다.



PLAN.md 파일 내용을 다 지워줘.
아래 내용에 대한 PLAN.md 을 새로 작성해줘

1. 맵은 2차원 Grid 형태이며, 테두리는 모두 값이 1 이야
2. 우주선 시작 좌표값은 (y, x) 기준으로 (1,1)로 해
3. 맵의 (1, 1) 값은 무조건 0 이어야만 해 (우주선 시작 위치)
4. 맵 테두리 내부영역은 랜덤하게 0과 1로 채워주는데 30%는 1이고 (장애물), 70%는 0 이었으면 해 (빈 공간)

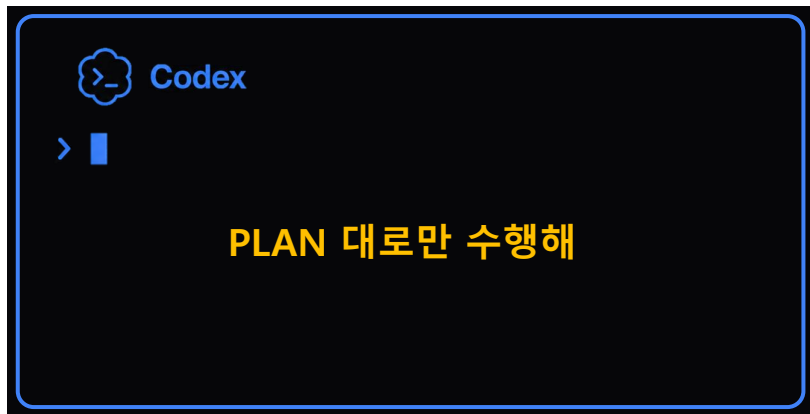
PLAN.md ×

PLAN.md > abc ## 파일 수정 대상

```
1  ## 맵 생성 규칙
2
3  1. 우주선 시작 좌표: `(y=1, x=1)`
4
5  2. 테두리(상/하/좌/우 가장자리)는 모두 `1` (장애물)
6     나머지 내부 영역은 기본 `0`
7
8  3. `(y=1, x=1)` 은 무조건 `0` (우주선 시작 위치 보장)
9
10 4. 내부 영역(테두리 제외)은 랜덤하게 채움
11   - `1` (장애물) : 30%
12   - `0` (빈 공간) : 70%
13   - 단, `(y=1, x=1)` 은 위 규칙과 무관하게 항상 `0`
14
15 ---
```

13. Action 수행 후 코드리뷰

- 만들어진 소스코드에 대해 모두 이해를 해야한다.
- 이해가 가지 않는 소스코드는 사용하면 안 된다.



```
@app.post("/init")
def init_grid(req: InitRequest):
    grid = []
    for y in range(req.height):
        row = []
        for x in range(req.width):
            # 테두리는 무조건 1
            if y == 0 or y == req.height - 1 or x == 0 or x == req.width - 1:
                row.append(1)
            # 우주선 시작 위치 (1,1)은 무조건 0
            elif y == 1 and x == 1:
                row.append(0)
            # 내부 영역: 30% 확률로 1, 70% 확률로 0
            else:
                row.append(1 if random.random() < 0.3 else 0)
        grid.append(row)

    return {"grid": grid, "rover": {"y": 1, "x": 1}}
```

14. Plan 작성 후, Plan 검토를 한다. 반드시 PLAN 내용을 모두 이해한다.



> PLAN.md 파일 내용을 모두 삭제하고 아래 내용으로 새로 작성해.

1. 백엔드

- 현재 생성된 맵과 우주선 상태를 조회하는 API를 하나 만들어.
- API는 새로운 맵을 생성하지 않고 현재 상태만 반환해.

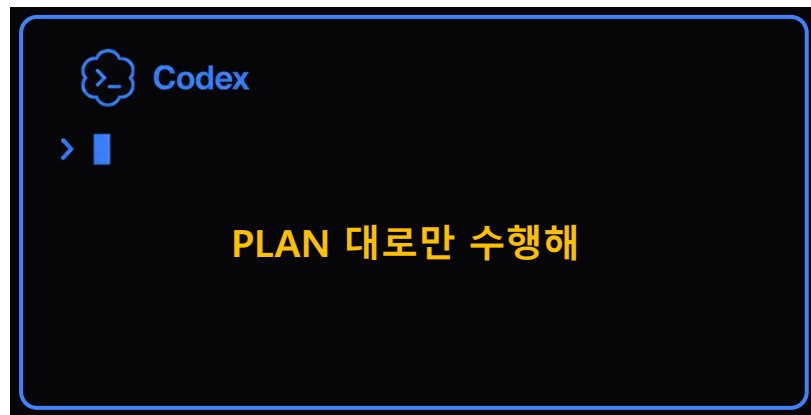
2. 프론트

- 맵 생성 후 0.5초마다 API를 호출해
- 맵은 유지하고 우주선 위치만 갱신해
- 우주선은 🚀 이모지로 표시해

```
PLAN.md x
PLAN.md > abc ## 상태 조회 및 실시간 출력 플랜 > abc ### API 명세
1  ## 상태 조회 및 실시간 출력 플랜
2
3  ### 기능 1 : 상태 조회 API (백엔드)
4
5  - `GET /state` 엔드포인트
6  - 현재 맵(grid), 우주선 좌표(rover)를 반환
7  - 응답: `{ "grid": [[int]], "rover": { "y": int, "x": int } }`
8
9  ---
10
11 ### 기능 2 : 실시간 상태 출력 (프론트엔드)
12
13 - 0.5초마다 `GET /state` API 호출 (폴링)
14 - 응답받은 grid와 rover 좌표를 화면에 렌더링
15 - 우주선은 이모지(🚀)로 표기
16 - `0` : 빈 셀 (□), `1` : 장애물 (■)
17
```


15. Action 수행 후, 코드리뷰를 한다.

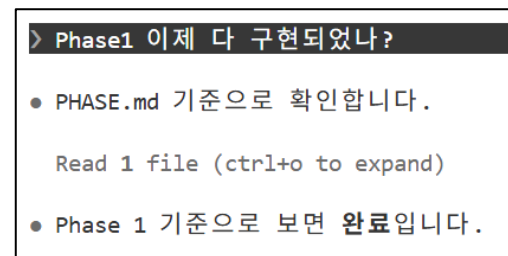
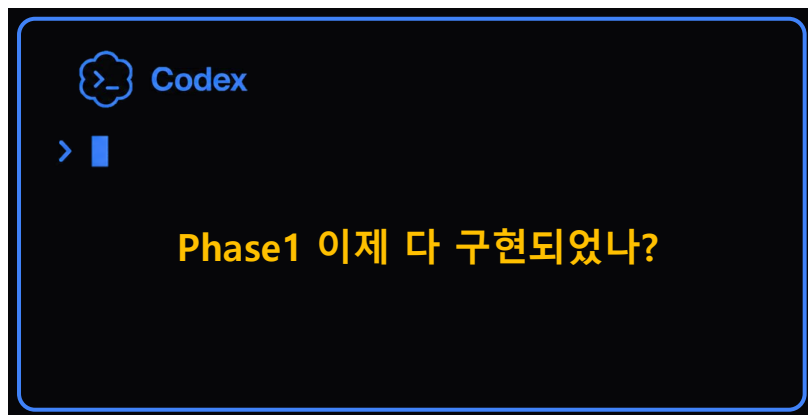
모든 코드가 이해가 가야 한다. (index.html은 관심사가 아니므로, 확인하지 않는다.)



```
        return {"grid": grid, "rover": game_state["rover"], "fuel": 0}

@app.get("/state")
def get_state():
    return {"grid": game_state["grid"], "rover": game_state["rover"]}
```

16. Phase 1이 종료되었는지 확인하자.



Chapter 05

방향키 개발 (Phase 2)

Agentic Engineering 실습

사전지식 Codex / Claude Code 의 장점

Claude Code의 장점

- 대규모 코드베이스 이해
- 의도 파악 및 검증을 더 잘함
- **PLAN** 수립

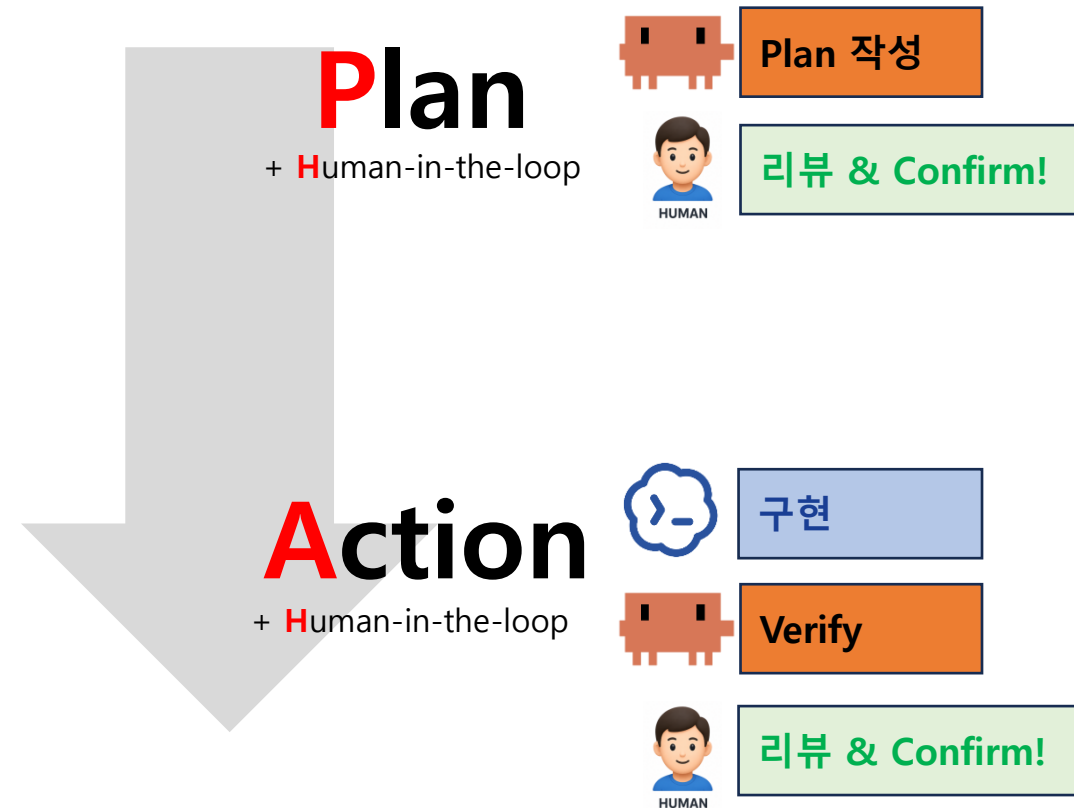
Codex의 장점

- **구현 성능과 속도가 빠름**
- 디버깅 성능이 좋음

Codex / Claude Code 2개 모델 동시에 사용하기

2개의 모델을 모두 사용하여

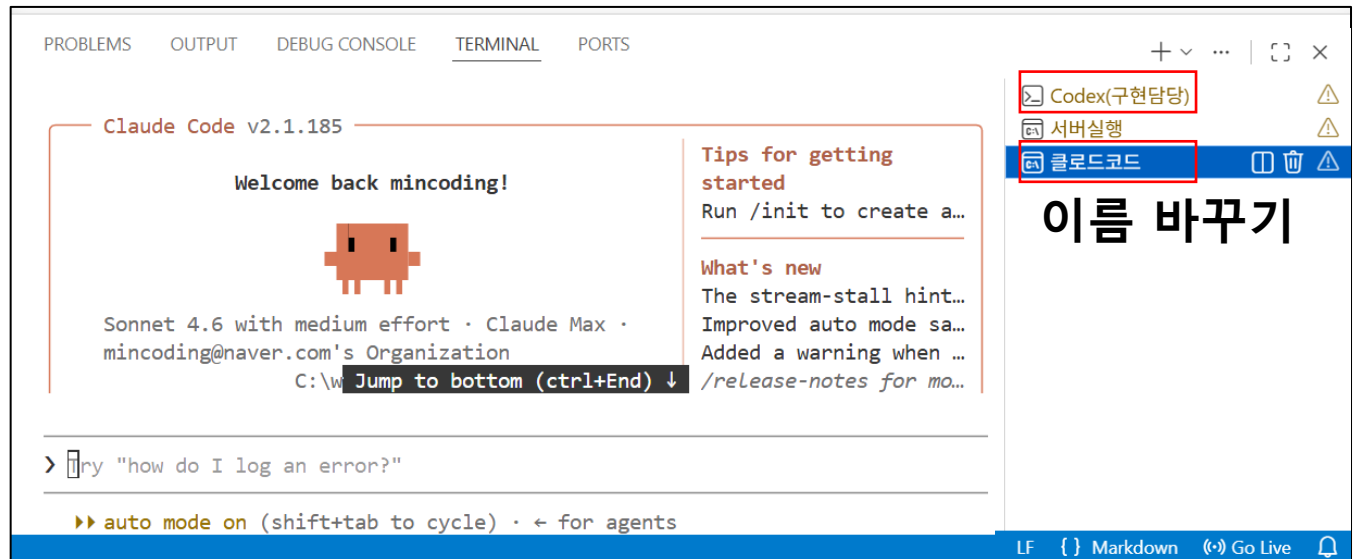
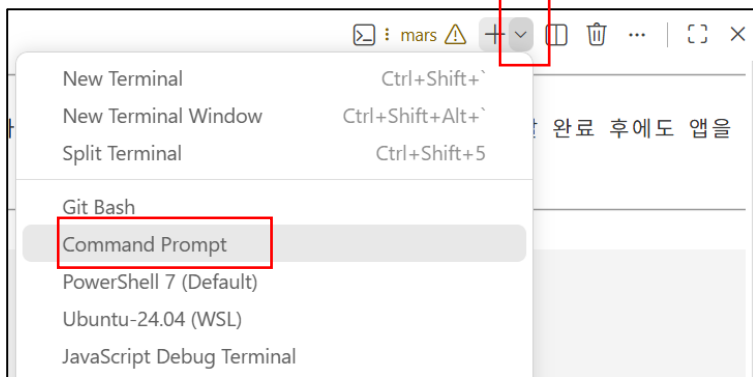
Harness + **Agentic Engineering** 동시 적용



터미널 창 하나 더 띄우기

Harness + Agentic Engineering 동시 적용을 위해
터미널 창을 하나 더 생성 후, 이름을 변경해준다.

- Codex 용도 : **구현**
- Claude Code 용도 : **PLAN 및 Verify**



Claude Code의 역할을 알림

17. CLAUDE.md 를 작성한다.

클로드 코드에서 실행!



▶ **CLAUDE.md를 루트 폴더에 파일 생성하고, AGENTS.md 파일 내용을 참고하여 적어줘**

Claude의 역할

- PLAN 작성
- PLAN대로 잘 구현되었는지 확인 후 PLAN대로 구현되도록 수정

PLAN 에 대한 초안은 다른 에이전트가 구현할 거야.

CLAUDE.md X

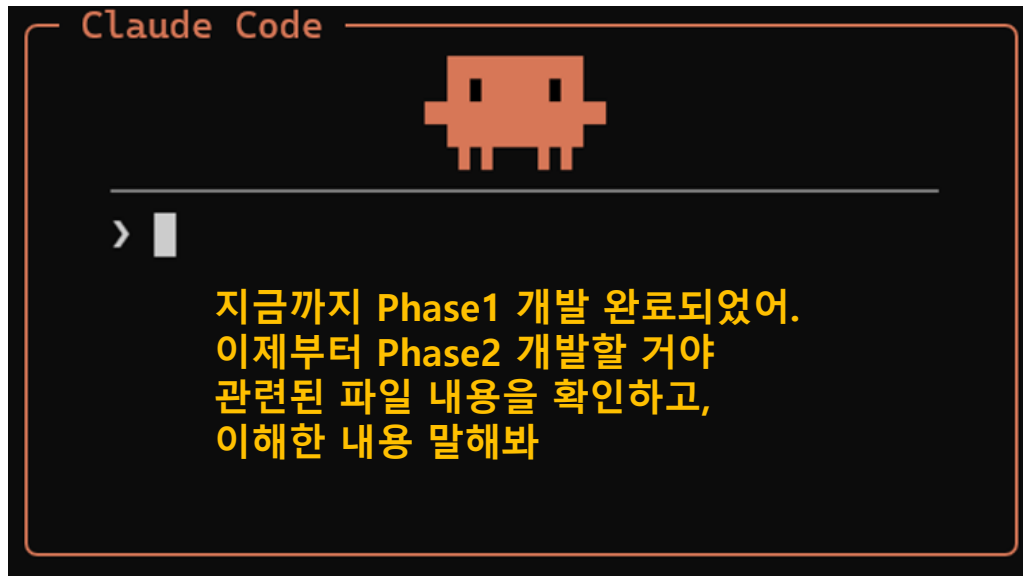
CLAUDE.md > abc # CLAUDE > abc ## 문서 구조

```
1  # CLAUDE
6  ## 기술 구성
8  - Frontend: index.html 단일 파일로 개발하며, HTML, CSS, JavaScript를 사용
9  ⚡
10 ## 문서 구조
11 - `docs/PHASES` 폴더에 단계별 개발 내용을 문서화합니다.
12 - `docs/PHASES/PHASE1.md`부터 `docs/PHASES/PHASE4.md`까지 각 단계 요구사항
13
14 ## Claude의 역할
15 1. 구현 전 `PLAN.md`를 작성합니다.
16 2. 구현 완료 후 `PLAN.md`에 정의된 내용대로 올바르게 구현되었는지 확인합니다.
17 3. PLAN과 다르게 구현된 부분이 있다면 PLAN에 맞게 수정합니다.
18
19 ## 작업 규칙
20 1. Claude는 직접 구현하지 않습니다.
21 2. 사용자가 시킨 것 외에는 아무것도 하지 않습니다.
22 3. 개발 완료 후 앱을 자동으로 실행하지 않습니다. 실행은 사용자가 직접 합니다.
23
```

Agent에게 현재 단계를 알려줌

18. Phase1 종료 및 Phase 2 시작을 알린다.

클로드 코드에서 실행!



> Phase2를 이제 시작할게
내용 요약해봐

- PHASE.md 기준 Phase 2 내용입니다.

방향키 명령 입력 후 이동

- 방향키(상/하/좌/우) 입력 → 백엔드로 전달
- 우주선이 해당 방향으로 이동
- 충돌이 없다면 **Pass** (정상 이동)
- 충돌이 있다면 **Warning** (이동 불가)

구현 전 확인할 것:

- 충돌 시 우주선은 **제자리** 유지인가요, 아니면 다른 처
- **Warning**은 화면에 텍스트로 표시하면 될까요?

19. 프론트엔드 개발만 지시한다.

프론트는 관심사가 아니므로, Plan 작성하지 않고 즉시 지시하자.

Codex에서 실행



Codex



Frontend에다가 키보드 방향키 같이
상,하,좌,우 명령을 줄 수 있는 버튼 UI를 만들어줘
그리고 TextBox와 '전송' 버튼도 하나씩 추가해줘

상,하,좌,우에 해당하는 버튼을 누르면
Text Box에 입력한 방향키가 입력돼
예를 들어 상,하,상,하 버튼을 누르면
"↑ ↓ ↑ ↓" 처럼 TextBox에 입력되는 것이지

전송 버튼을 누르더라도 API 동작 안 되도록 해줘

Height: 10 Width: 10 탐사준비

↑

← ↓ →

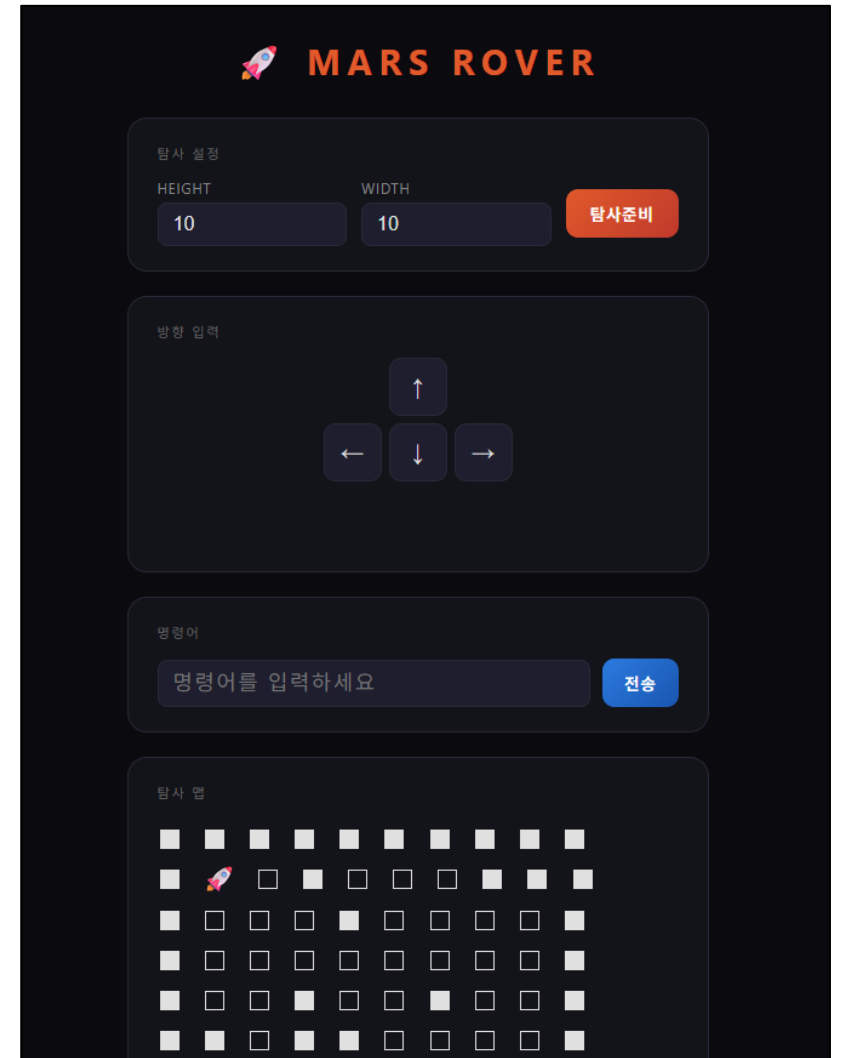
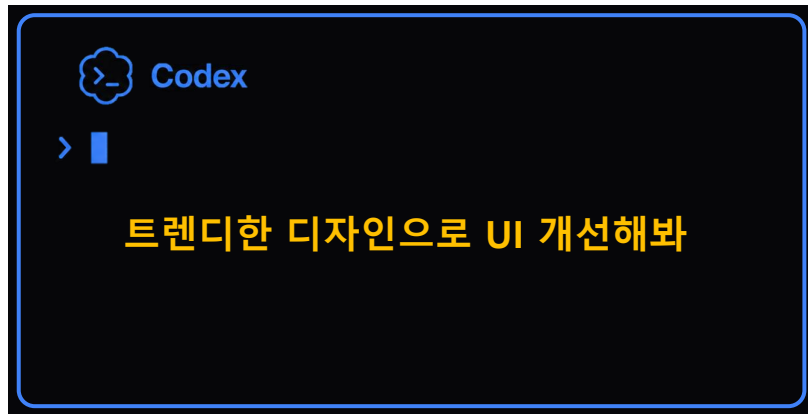
↑←↓→↑←↓ 전송

■	■	■	■	■	■	■	■	■	■
■	🚀	□	■	□	□	□	■	■	■
■	□	□	□	■	□	□	□	□	■
■	□	□	□	□	□	□	□	□	■
■	□	□	■	□	□	■	□	□	■
■	■	□	■	■	□	□	□	□	■
■	□	□	□	■	□	■	□	□	■
■	□	□	■	□	□	□	■	□	■
■	□	■	□	□	■	■	□	□	■
■	■	■	■	■	■	■	■	■	■

20. UI 개선 요청

조금 더 이쁘게 개선시켜보자.

프론트는 관심사가 아니므로, 코드리뷰를 하지 않는다.



21. PLAN 작성

방향대로 이동하여 값(state)를 수정하는 구현에 대한 PLAN 작성 요청

클로드 코드에서 실행!

Claude Code



PLAN.md 파일 내용을 다 지워줘.
아래 내용에 대한 PLAN.md를 새로 작성해줘

먼저 index.html 파일을 다시 읽어서,
내용 확인해봐. UI 코드 바꾸었어

1) 방향키 명령어 배열을 입력받는 API를 하나 만들어
입력된 방향키대로 한 칸씩 우주선 좌표를 이동시켜줘

2) 방향대로 한 칸 씩 움직이는데, 벽에 부딪히면
더 이상 진행하지 말고, 움직임을 종료해줘

3) UI에서 '전송' 버튼을 누르면
그동안 입력했던 방향키 명령어들이 백엔드로 전송된다.

```
PLAN.md x
PLAN.md > abc ## Phase 2 구현 플랜 > abc ### API 명세

1  ## Phase 2 구현 플랜
2
3  ### 기능 1 : 방향키 명령어 전송 API (백엔드)
4
5  - `POST /move` 엔드포인트
6  - 요청 body: `{ "commands": "↑↓←→" }` (문자열)
7  - 명령어를 한 글자씩 순서대로 처리
8  - 각 명령어마다 우주선 좌표를 한 칸씩 이동
9  - `↑` : y - 1
10 - `↓` : y + 1
11 - `←` : x - 1
12 - `→` : x + 1
13 - 응답: `{ "rover": { "y": int, "x": int } }`
14
15 ---
16
17 ### 기능 2 : 벽 충돌 시 이동 종료
18
19 - 다음 칸이 `1` (장애물) 이면 이동하지 않고 명령 처리 종료
20 - 이후 남은 명령어는 실행하지 않음
21 - 응답에 충돌 여부 포함: `{ "rover": { "y": int, "x": int }, "status": "pas`
22
23 ---
24
25 ### API 명세
26
27 | Method | Endpoint | 요청 body | 응답 |
```

22. PLAN.md 대로 구현 초안 작성을 시킨다.
여기서 완성한 코드는 아직 검토를 안해도 좋다.

Codex에서 실행



Codex

> █

PLAN.md 파일을 읽고
PLAN 대로 구현 시작해

23. 인간의 리뷰 전, Verify 를 수행시킨다.

이후 리뷰를 수행한다. 만들어진 소스코드에 대해 모두 이해를 해야한다.

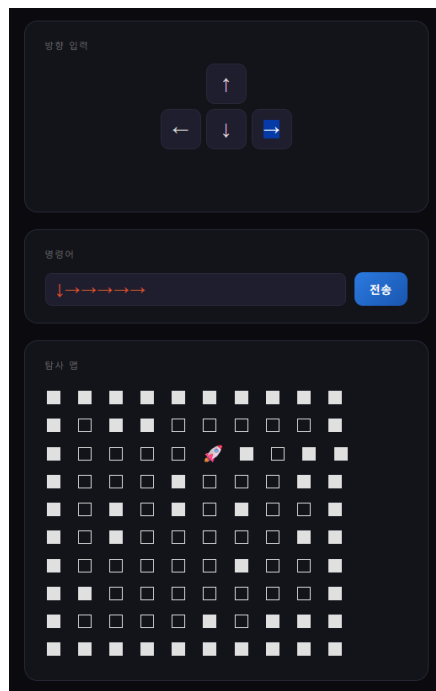
- 이해가 가지 않는 소스코드는 사용하면 안 된다.

클로드 코드에서 실행!



> █

PLAN.md 문서대로
잘 구현되었는지 확인 후,
부족한 부분은 코드를 개선해줘



```
main.py x
main.py > serve_index
56
57 DIRECTION_MAP = {
58     "↑": (-1, 0),
59     "↓": (1, 0),
60     "←": (0, -1),
61     "→": (0, 1),
62 }
63
64
65 @app.post("/move")
66 def move_rover(req: MoveRequest):
67     grid = game_state["grid"]
68     rover = game_state["rover"]
69     y, x = rover["y"], rover["x"]
70
71     for cmd in req.commands:
72         dy, dx = DIRECTION_MAP.get(cmd, (0, 0))
73         ny, nx = y + dy, x + dx
74
75         # 다음 칸이 장애물이면 이동 종료
76         if grid[ny][nx] == 1:
77             game_state["rover"] = {"y": y, "x": x}
78             return {"rover": game_state["rover"], "status": "warning"}
79
80         y, x = ny, nx
81
82     game_state["rover"] = {"y": y, "x": x}
83     return {"rover": game_state["rover"], "status": "pass"}
84
```

24. PLAN 작성

장애물에 부딪칠 때 처리, 플랜 요청

클로드 코드에서 실행!



PLAN.md 파일 내용을 다 지워줘.
아래 내용에 대한 PLAN.md 을 새로 작성해줘

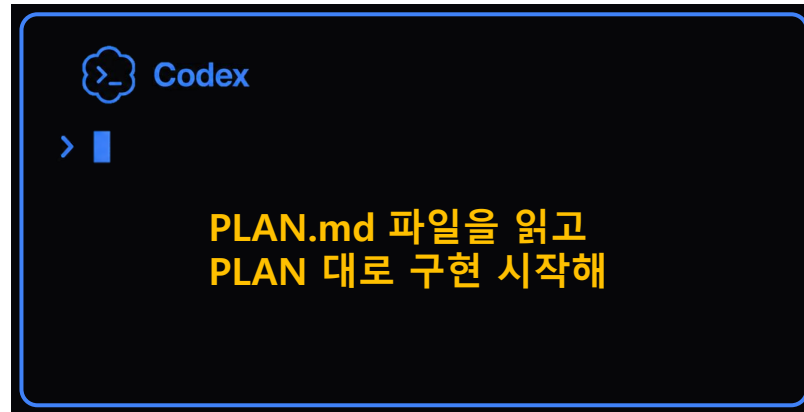
- 1) 벽 충돌 시 더 이상 움직이지 않고, Warning을 반환
- 2) 벽 충돌 없이 이동 시, Pass를 반환

프론트엔드에서는 '전송' 이후 결과를 표현한다.
(명령어 결과 Pass / Warning)

```
PLAN.md x
PLAN.md > abc ## Phase 2 - 충돌 결과 표시 플랜 > abc ### 파일 수정 대상
1  ## Phase 2 - 충돌 결과 표시 플랜
2
3  ### 기능 1 : 충돌 결과 반환 (백엔드)
4
5  - 벽 충돌 시 이동 중단 후 `status: "warning"` 반환
6  - 충돌 없이 모든 명령 완료 시 `status: "pass"` 반환
7
8  ---
9
10 ### 기능 2 : 명령 결과 표시 (프론트엔드)
11
12 - 전송 버튼 클릭 후 `/move` 응답의 `status` 값을 화면에 표시
13   - `pass` : 정상 이동 메시지 표시
14   - `warning` : 충돌 경고 메시지 표시
15
16 ---
17
18 ### API 명세
19
20 | Method | Endpoint | 요청 body | 응답 |
21 |-----|-----|-----|-----|
22 | POST | `/move` | `{ "commands": str }` | `{ "rover"
```

25. PLAN.md 대로 구현 초안 작성을 시킨다.
여기서 완성한 코드는 아직 검토를 안해도 좋다.

Codex에서 실행



26. 인간의 리뷰 전, Verify 를 수행시킨다.

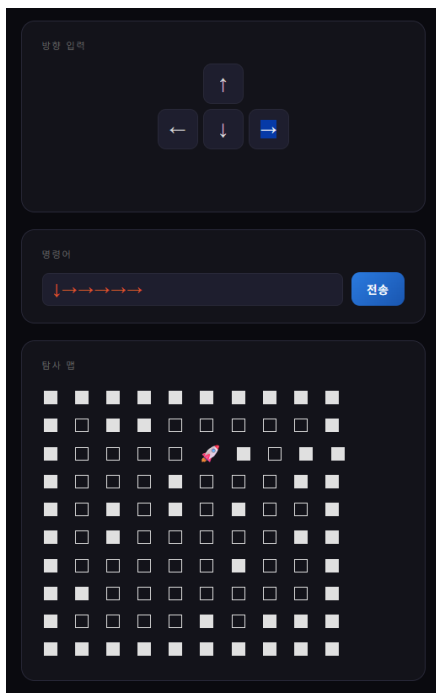
이후 리뷰를 수행한다. 만들어진 소스코드에 대해 모두 이해를 해야한다.

- 이해가지 않는 소스코드는 사용하면 안된다.

클로드 코드에서 실행!



PLAN.md 문서대로
잘 구현되었는지 확인 후,
부족한 부분은 코드를 개선해줘



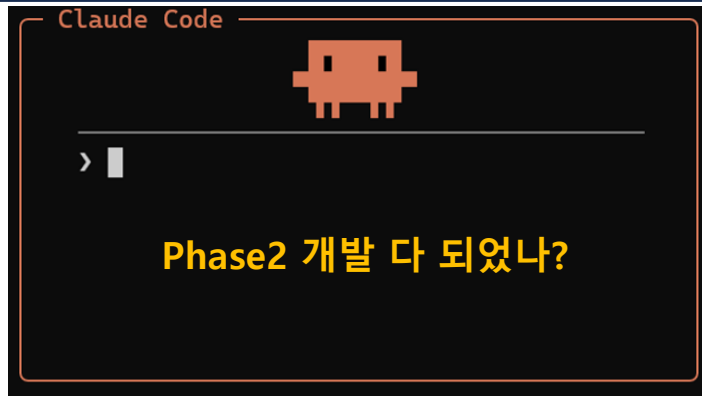
```
main.py x
main.py > serve_index
56
57 DIRECTION_MAP = {
58     "↑": (-1, 0),
59     "↓": (1, 0),
60     "←": (0, -1),
61     "→": (0, 1),
62 }
63
64
65 @app.post("/move")
66 def move_rover(req: MoveRequest):
67     grid = game_state["grid"]
68     rover = game_state["rover"]
69     y, x = rover["y"], rover["x"]
70
71     for cmd in req.commands:
72         dy, dx = DIRECTION_MAP.get(cmd, (0, 0))
73         ny, nx = y + dy, x + dx
74
75         # 다음 칸이 장애물이면 이동 종료
76         if grid[ny][nx] == 1:
77             game_state["rover"] = {"y": y, "x": x}
78             return {"rover": game_state["rover"], "status": "warning"}
79
80         y, x = ny, nx
81
82     game_state["rover"] = {"y": y, "x": x}
83     return {"rover": game_state["rover"], "status": "pass"}
84
```


Action 수행 후 검토

27. 모두 개발이 끝났는지 확인해보자.

만약 다 수행되지 않았다면, Claude에게 PLAN을 세우고, Codex로 구현을 시키고, 다시 Claude에게 Verify를 지시한다.

클로드 코드에서 실행!



충돌 있으면 Pass	✓
충돌 있으면 Warning	✓
결과 UI 표시	✓

Phase 2 완료입니다. Phase 3으로 넘어갈까요?

Chapter 06

연료소모량 기능추가 (Phase 3)

Agentic Engineering 실습

Agentic Engineering 원칙 - 리뷰 가능한 수준으로 지시를 해야 한다.

Plan 작성의 범위

AI가 만든 코드를, 사람이 모두 리뷰해야 한다.

따라서 리뷰 가능한 수준으로 Plan을 잡고, 구현해야 한다.

Agentic Engineering을 하기 위해서는 개발 지식이 필수이다.

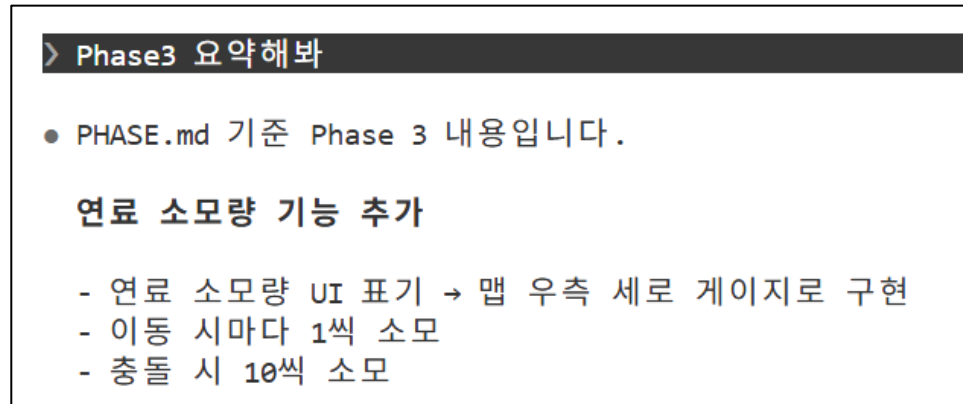
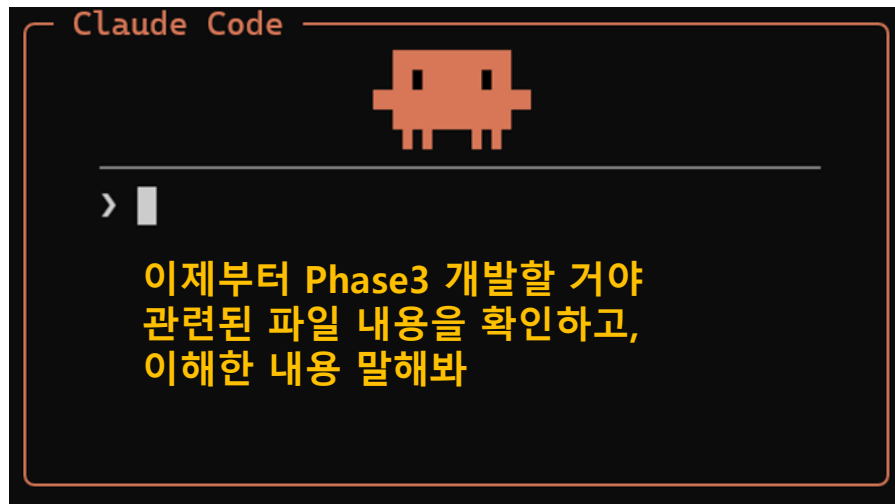
개발 지식이 없다면, 이 Plan이 어느 정도 범위로 구현될지 예상할 수 없기에, 개발 지식 없이 Agentic Engineering을 하는 것은 불가능하다.



너무 과도한 Plan 작성 시, 결과물이 광범위하기 때문에 사람이 리뷰하기 어렵다.
따라서 리뷰 가능한 수준으로 Plan을 세워야 한다.

Agent에게 현재 단계를 알려줌

28. Phase 3 시작을 알린다.



29. 프론트 엔드 개발만 지시한다.

프론트는 관심사가 아니므로, Plan 작성하지 않고 즉시 지시하자.

Codex에서 실행



Codex

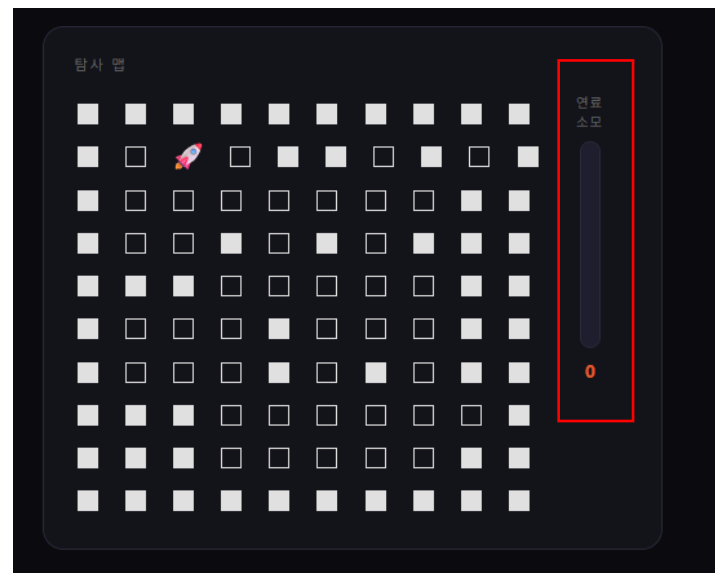


Frontend에다가

맵 우측에 연료소모량을 나타내는 UI 표기해줘

게이지의 기본값은 0 이고,
0 ~ 100 까지 값을 나타낼 수 있어

UI 외 아무 기능을 만들지 말아줘



30. PLAN 작성

연료소모량 관련 코드 플랜 작성

클로드 코드에서 실행!

Claude



기존 PLAN.md를 다 지우고,
다시 아래 내용에 대한 PLAN.md를 작성해줘

먼저 index.html이 변경되었으니, 다시 파일을 읽어 내용 확인해줘
연료소모량 UI 코드를 확인해봐

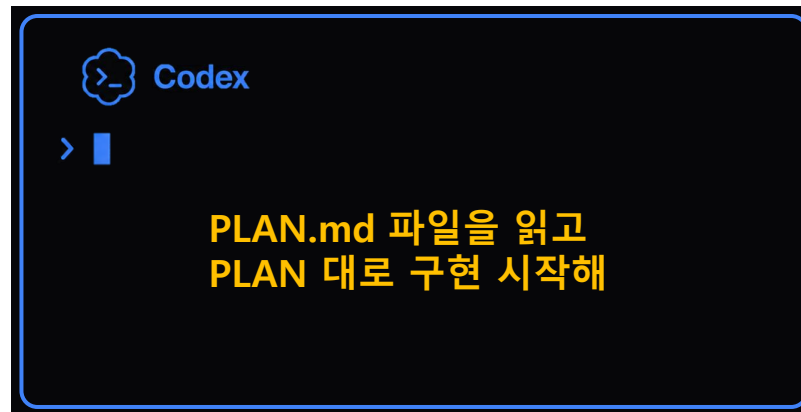
- 1) 연료게이지는 0 ~ 100 까지 값을 가질 수 있어
- 2) 연료소모량 상태를 기본값 0으로 만들어
- 3) 한 번 이동 시마다 1씩 소요되고
- 4) 충돌 발생 시 10씩 소모되도록 해

그리고 게임 상태를 반환하는 API에는
맵, 우주선 위치 뿐만 아니라, 현재 연료소모량도
반환하도록 수정하고
프론트엔드에서는 연료소모량 값을 UI에 표시해주도록 해

```
PLAN.md X
PLAN.md > abc ## Phase 2 - 충돌 결과 표시 플랜 > abc ### 파일 수정 대상
1  ## Phase 2 - 충돌 결과 표시 플랜
2
3  ### 기능 1 : 충돌 결과 반환 (백엔드)
4
5  - 벽 충돌 시 이동 중단 후 `status: "warning"` 반환
6  - 충돌 없이 모든 명령 완료 시 `status: "pass"` 반환
7
8  ---
9
10 ### 기능 2 : 명령 결과 표시 (프론트엔드)
11
12 - 전송 버튼 클릭 후 `/move` 응답의 `status` 값을 화면에 표시
13   - `pass` : 정상 이동 메시지 표시
14   - `warning` : 충돌 경고 메시지 표시
15
16 ---
17
18 ### API 명세
19
20 | Method | Endpoint | 요청 body | 응답 |
21 |-----|-----|-----|-----|
22 | POST | `/move` | `{ "commands": str }` | `{ "rover"
```

31. PLAN.md 대로 구현 초안 작성을 시킨다.
여기서 완성한 코드는 아직 검토를 안해도 좋다.

Codex에서 실행



32. 인간의 리뷰 전, Verify 를 수행시킨다.

이후 리뷰를 수행한다. 만들어진 소스코드에 대해 모두 이해를 해야한다.

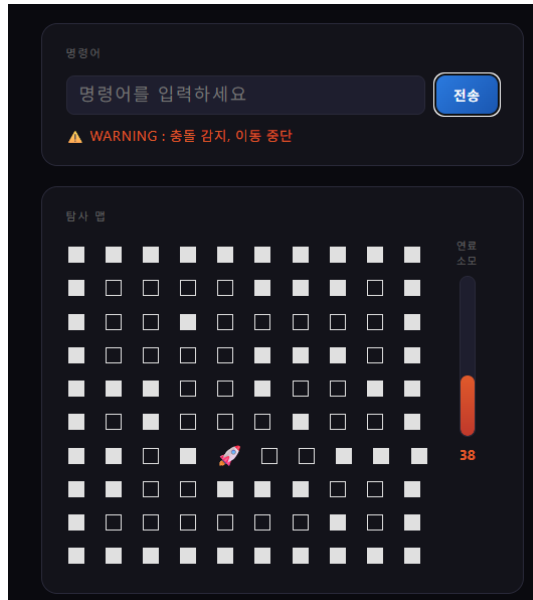
- 이해가지 않는 소스코드는 사용하면 안된다.

클로드 코드에서 실행!



> █

PLAN.md 문서대로
잘 구현되었는지 확인 후,
부족한 부분은 코드를 개선해줘



```
@app.post("/move")
def move_rover(req: MoveRequest):
    grid = game_state["grid"]
    rover = game_state["rover"]
    y, x = rover["y"], rover["x"]

    for cmd in req.commands:
        dy, dx = DIRECTION_MAP.get(cmd, (0, 0))
        ny, nx = y + dy, x + dx

        if grid[ny][nx] == 1:
            game_state["fuel"] += 10 # 충돌 시 10 소모
            game_state["rover"] = {"y": y, "x": x}
            return {"rover": game_state["rover"], "status": "warning", "fuel": game_state["fuel"]}

        game_state["fuel"] += 1 # 이동 성공 시 1 소모
        y, x = ny, nx

    game_state["rover"] = {"y": y, "x": x}
    return {"rover": game_state["rover"], "status": "pass", "fuel": game_state["fuel"]}
```

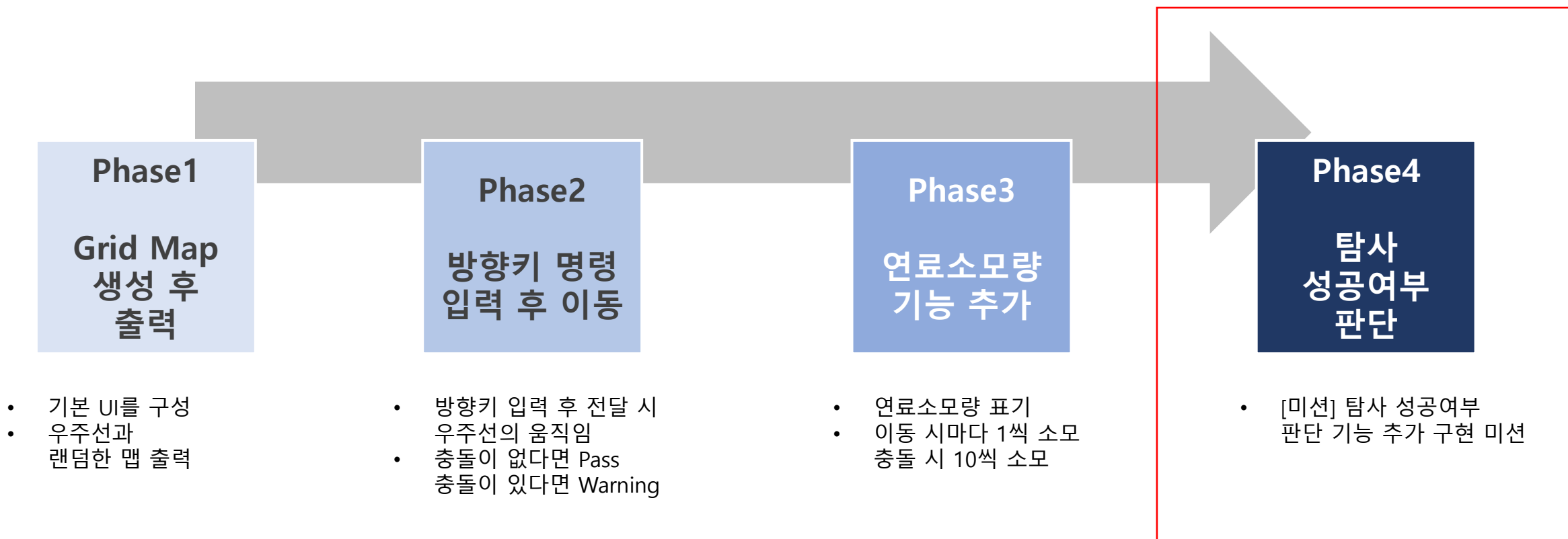

Chapter 07

미션 (Phase 4)

Agentic Engineering 실습

앞으로 남은 Phase

마지막 한 Phase가 남아있다.
탐사 성공 여부 판단

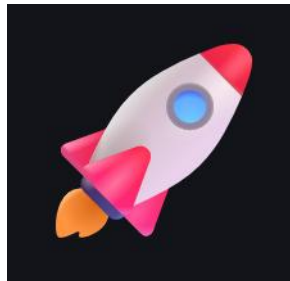


[도전] 남은 Phase 개발하기

Agentic Engineering을 적용하여 구현한다.

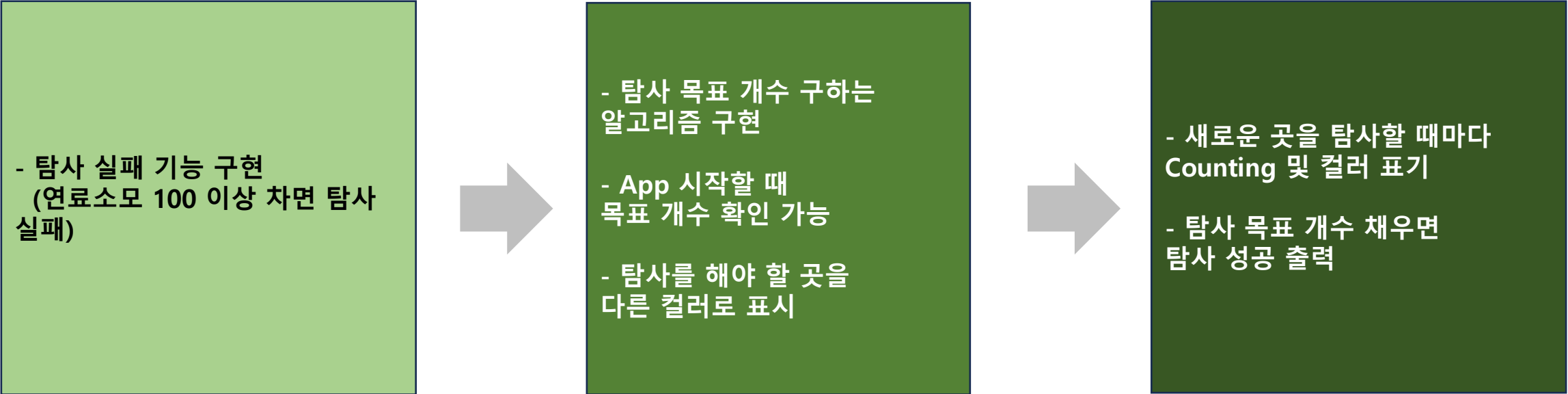
Phase4 : 탐사 성공여부 판단

탐험할 수 있는 모든 곳을 탐사를 완료한 경우, "탐사 성공" 메시지가 뜬다.
연료소모량을 모두 소진하는 경우, "탐사 실패" 메시지가 뜬다.



해결방법

아래 Step으로 나누면 더 정확한 구현이 가능하다.



- 탐사 실패 기능 구현
(연료소모 100 이상 차면 탐사 실패)

- 탐사 목표 개수 구하는 알고리즘 구현
- App 시작할 때 목표 개수 확인 가능
- 탐사를 해야 할 곳을 다른 컬러로 표시

- 새로운 곳을 탐사할 때마다 Counting 및 컬러 표기
- 탐사 목표 개수 채우면 탐사 성공 출력

Agentic Engineering 을 수행했다면?

다음 질문에 대해 대답할 수 있어야 합니다.

Agentic Engineering으로 소스코드와 원리가 머릿속에 남아 있어야 합니다.

Q1. 탐사 전체 완료 여부를 파악하는 방법은 어떤 원리입니까?

Q2. 탐사 진행을 했던 곳과 안 한 곳은
어떤 방식으로 맵 배열에 기록되고 있습니까?

만약 코드리뷰를 하지 않았다면, 아래 질문에 대답하기 어렵습니다.

고품질의 SW 확보를 위해, 사람의 코드리뷰는 필수입니다.

감사합니다.